บทที่ 13: การคงอยู่ของข้อมูลด้วย SQLite (Data Persistence with SQLite)

13.1 บทนำ: การบันทึกข้อมูลไว้ใช้ภายหลัง

ลองนึกถึงเหตุการณ์ที่น่าหงุดหงิดที่สุดของการใช้แอปบนมือถือ เราพิมพ์โน้ตเรื่องสำคัญไว้ยืดยาว หรือเล่นเกมจนทำคะแนนสูงสุดเป็นสถิติใหม่ แล้วจู่ ๆ ก็เผลอปัดปิดแอป หรือแบตเตอรี่หมดไปเฉย ๆ พอชาร์จแบตแล้วเปิดแอปขึ้นมาใหม่ หน้าจอกลับว่างเปล่า ทุกอย่างหายไปเหมือนไม่เคยเกิดขึ้น

สิ่งที่แอปสร้างให้เราเห็นมาตลอดคือภาพลวงตาของความคงอยู่ เราพิมพ์ปุ๊บก็เห็นข้อมูลบนจอปั๊บ จึงรู้สึกว่าข้อมูลปลอดภัยแล้ว แต่ความจริงข้อมูลนั้นลอยอยู่ในหน่วยความจำหลัก (RAM) ซึ่งถูกออกแบบมาให้ทำงานเร็ว แลกกับข้อเสียสำคัญคือต้องมีไฟเลี้ยงตลอดเวลา และระบบปฏิบัติการพร้อมจะเรียกคืนพื้นที่ของแอปที่อยู่เบื้องหลังได้ทุกเมื่อเพื่อนำไปให้แอปอื่นใช้ งานของบทนี้คือย้ายข้อมูลจากพื้นที่ชั่วคราวนั้นลงไปเก็บในที่ที่อยู่รอดข้ามการปิดแอปได้จริง

จนถึงตอนนี้ แอปพลิเคชันของเราสามารถแสดง UI, จัดการ state ชั่วคราวภายในวิดเจ็ตเดียว (ephemeral state) ด้วย setState, จัดการ state ที่ใช้ร่วมกันทั้งแอป (app state) ด้วย Provider, และดึงข้อมูลจากอินเทอร์เน็ตได้แล้ว. แต่จะเกิดอะไรขึ้นเมื่อผู้ใช้ปิดแอป? state ทั้งสองแบบอยู่ในหน่วยความจำ (RAM) ของแอปเท่านั้น ข้อมูลทุกอย่าง เช่น ค่าตัวนับ หรือข้อมูลที่ดึงมาจาก API จะหายไปพร้อมกับแอป. จะทำอย่างไรถ้าเราต้องการ:

- บันทึกข้อมูลของผู้ใช้: เช่น รายการสิ่งที่ต้องทำ (to-do list), บันทึกส่วนตัว, การตั้งค่าแอป, หรือความคืบหน้าในเกม
- ทำงานแบบออฟไลน์: เก็บข้อมูลที่ดึงมาจากเครือข่ายไว้ เพื่อให้ผู้ใช้ยังสามารถดูข้อมูลบางส่วนได้แม้ไม่มีการเชื่อมต่ออินเทอร์เน็ต
- ปรับปรุงประสิทธิภาพ: แคชข้อมูลที่เข้าถึงบ่อยๆ ไว้ในเครื่อง เพื่อลดการเรียกเครือข่ายที่ไม่จำเป็น

สำหรับสถานการณ์เหล่านี้ เราต้องการ การคงอยู่ของข้อมูล (Data Persistence) – วิธีการบันทึกข้อมูลลงในที่จัดเก็บถาวรบนอุปกรณ์ของผู้ใช้ เพื่อให้ข้อมูลยังคงอยู่แม้แอปจะถูกปิดและเปิดใหม่.

บทที่ 12 ปิดท้ายไว้ด้วยสถานการณ์ของผู้ใช้ที่เปิดแอปในลิฟต์แล้วเห็นแต่ข้อความว่าเชื่อมต่อไม่ได้ ทั้งที่ข้อมูลชุดเดียวกันเพิ่งเดินทางมาถึงเครื่องเมื่อไม่กี่ชั่วโมงก่อน พร้อมคำถามสามข้อ คือข้อมูลชุดนั้นควรเก็บไว้ที่ไหนและใครเป็นผู้เก็บ เมื่อมีข้อมูลสองชุดชุดใดคือความจริง และสามสถานะที่ใช้มาทั้งบทยังพอหรือไม่ บทนี้เริ่มจากการสร้างที่เก็บข้อมูลบนตัวเครื่องให้ใช้งานได้จริงก่อน แล้วจึงกลับมาตอบคำถามทั้งสามข้ออย่างครบถ้วนในหัวข้อ 13.10

มีหลายวิธีในการทำ data persistence ใน Flutter, ตั้งแต่การใช้ shared_preferences สำหรับข้อมูล key-value ง่ายๆ [1], การเขียนลงไฟล์โดยตรง, ไปจนถึงการใช้ฐานข้อมูลแบบ NoSQL บนคลาวด์เช่น Firebase Firestore. อย่างไรก็ตาม สำหรับข้อมูลที่มีโครงสร้างและมีความสัมพันธ์กันซึ่งต้องการเก็บไว้ในเครื่อง SQLite เป็นตัวเลือกที่ทรงพลังและใช้กันอย่างแพร่หลาย.

SQLite เป็นระบบจัดการฐานข้อมูลเชิงสัมพันธ์ (Relational Database Management System - RDBMS) แบบ file-based ที่มีน้ำหนักเบาและฝังตัวได้ มันถูกใช้ในแอปพลิเคชันมือถือนับไม่ถ้วน (ทั้ง Android และ iOS มีการสนับสนุน SQLite ในตัว).

ในบทนี้ เราจะเรียนรู้วิธีใช้ SQLite ใน Flutter เพื่อสร้าง, อ่าน, อัปเดต, และลบ (CRUD - Create, Read, Update, Delete) ข้อมูลที่มีโครงสร้าง [2]. เราจะใช้แพ็กเกจยอดนิยมสองตัว:

- sqflite: แพ็กเกจหลักสำหรับโต้ตอบกับฐานข้อมูล SQLite ใน Flutter. [3]
- path_provider: แพ็กเกจสำหรับค้นหาตำแหน่งที่ถูกต้องบนระบบไฟล์ของอุปกรณ์เพื่อจัดเก็บไฟล์ฐานข้อมูล SQLite. [4]

เราจะสร้าง Database Helper Class ซึ่งเป็นรูปแบบการออกแบบทั่วไปที่ห่อหุ้มตรรกะการโต้ตอบกับฐานข้อมูลทั้งหมดไว้ในที่เดียว ทำให้โค้ดของเราเป็นระเบียบและจัดการง่ายขึ้น. จากกล่าวมาข้างต้น ในบทนี้ เราจะเรียนรู้แนวทางการคงอยู่ของข้อมูล โดยมีเนื้อหาครอบคลุมดังนี้:

- แนวคิดพื้นฐานของ SQLite และฐานข้อมูลเชิงสัมพันธ์.
- การเพิ่ม dependencies ที่จำเป็น.

- การสร้าง Database Helper class (มักใช้รูปแบบ Singleton).
- การเปิดฐานข้อมูลและสร้างตารางโดยใช้คำสั่ง SQL CREATE TABLE.
- การดำเนินการ CRUD พื้นฐาน: insert, query, update, delete.
- การแปลงข้อมูลระหว่างรูปแบบ Map (ที่ sqflite ใช้) และ Dart model objects ของเรา.
- การแปลงข้อมูลชนิดที่ SQLite ไม่มี เช่น bool, DateTime และ enum ให้เก็บได้อย่างถูกต้อง.
- การเปลี่ยนโครงสร้างตารางหลังจากแอปถูกใช้งานจริงแล้วด้วย onUpgrade โดยไม่ทำให้ข้อมูลของผู้ใช้หาย.
- การรวมหลายคำสั่งให้สำเร็จหรือล้มเหลวไปพร้อมกันด้วย transaction และ batch.
- การใช้ฐานข้อมูลในเครื่องคู่กับข้อมูลจากเครือข่าย เพื่อให้แอปเปิดดูข้อมูลได้แม้ไม่มีสัญญาณ.

เมื่อจบบทนี้ คุณจะสามารถสร้างฐานข้อมูล SQLite ภายในแอป Flutter ของคุณ, กำหนดโครงสร้างตาราง, และดำเนินการจัดการข้อมูลพื้นฐานได้ ซึ่งเป็นทักษะที่จำเป็นสำหรับการสร้างแอปพลิเคชันที่ซับซ้อนและทำงานได้จริง.

13.2 SQLite คืออะไร?

SQLite เป็นไลบรารีซอฟต์แวร์ที่ implement ระบบจัดการฐานข้อมูล SQL แบบ self-contained, serverless, zero-configuration, และ transactional [5].

- Self-contained: ไม่ต้องการ external dependencies มากนัก.
- Serverless: ไม่มีการทำงานของเซิร์ฟเวอร์แยกต่างหาก ฐานข้อมูลทั้งหมด (คำจำกัดความ, ตาราง, ดัชนี, และข้อมูลเอง) ถูกเก็บไว้ในไฟล์ธรรมดาไฟล์เดียวบนอุปกรณ์โฮสต์.
- Zero-configuration: ไม่จำเป็นต้องตั้งค่าหรือบริหารจัดการเหมือนฐานข้อมูลเซิร์ฟเวอร์ขนาดใหญ่.
- Transactional: การดำเนินการทั้งหมดเป็นไปตามหลัก ACID (Atomicity, Consistency, Isolation, Durability) ทำให้มั่นใจในความน่าเชื่อถือของข้อมูล.
- SQL Database Engine: ใช้ภาษา SQL (Structured Query Language) มาตรฐานสำหรับการกำหนดและจัดการข้อมูล.

เนื่องจากลักษณะที่เป็น file-based และมี footprint ขนาดเล็ก SQLite จึงเหมาะอย่างยิ่งสำหรับอุปกรณ์เคลื่อนที่และแอปพลิเคชันแบบฝังตัว. Flutter ใช้ประโยชน์จากการสนับสนุน SQLite ที่มีอยู่แล้วบน Android และ iOS ผ่านแพ็กเกจ sqflite.

เลือกที่เก็บให้เหมาะกับข้อมูล

SQLite ไม่ใช่คำตอบของข้อมูลทุกชนิด ก่อนเลือกเครื่องมือให้ถามก่อนว่าข้อมูลมีรูปร่างแบบไหนและต้องค้นหาอย่างไร

- shared_preferences เหมาะกับค่าเดี่ยว ๆ จำนวนไม่มาก ที่อ่านด้วยชื่อคีย์ เช่นธีมมืดหรือสว่าง ภาษาที่ผู้ใช้เลือก หรือเคยเปิดหน้าแนะนำแอปแล้วหรือยัง ไม่เหมาะกับรายการที่ยาวขึ้นเรื่อย ๆ เพราะค้นหาหรือเรียงลำดับไม่ได้
- ไฟล์ เหมาะกับข้อมูลก้อนใหญ่ที่อ่านทั้งก้อน เช่นรูปภาพ เอกสาร หรือ JSON ที่ได้จากเซิร์ฟเวอร์แล้วต้องการเก็บไว้ทั้งชุด
- SQLite เหมาะกับข้อมูลที่เป็นรายการหลายแถวมีโครงสร้างเหมือนกัน ต้องค้นหา กรอง เรียงลำดับ หรือแก้ไขทีละแถว เช่นบันทึกส่วนตัว รายการรายรับรายจ่าย หรือข้อมูลที่ดึงมาจากเซิร์ฟเวอร์เพื่อเปิดดูตอนออฟไลน์

ลองเทียบให้เห็นภาพว่าทำไมไม่เขียนทุกอย่างต่อท้ายลงไฟล์ข้อความธรรมดา ถ้าข้อมูลมีแค่ชื่อกับนามสกุลสองบรรทัด ไฟล์ข้อความก็เพียงพอ แต่แอปบัญชีรายรับรายจ่ายที่มีรายการหลายพันรายการเป็นอีกเรื่องหนึ่ง เมื่อผู้ใช้ต้องการแก้ยอดเงินของรายการเมื่อสามเดือนก่อน แอปต้องอ่านข้อความทั้งไฟล์ขึ้นมา ค้นหาบรรทัดนั้นให้เจอ แก้ แล้วเขียนทับไฟล์เดิมทั้งไฟล์ ช้าและเปลืองทรัพยากร และถ้าแอปถูกปิดกลางจังหวะเขียนทับ ไฟล์อาจเสียทั้งไฟล์ ฐานข้อมูลอย่าง SQLite ถูกออกแบบมาเพื่อแก้ปัญหานี้โดยตรง คือแก้เฉพาะแถวที่ต้องการ และรับประกันว่าการเขียนแต่ละครั้งจะสำเร็จครบหรือไม่เกิดขึ้นเลย

ข้อควรรู้อีกข้อหนึ่งคือ SQLite มีชนิดข้อมูลให้เลือกน้อยกว่าภาษา Dart มาก ไม่มีชนิด boolean และไม่มีชนิดวันที่ [6] แพ็กเกจ sqflite รับค่าได้เพียงตัวเลข

ข้อความ และข้อมูลไบนารีเท่านั้น [7] ข้อมูลชนิดอื่นต้องแปลงก่อนบันทึกเสมอ รายละเอียดอยู่ในหัวข้อ 13.7

13.3 แพ็กเกจที่จำเป็น: sqflite และ path_provider

ในการทำงานกับ SQLite ใน Flutter เราต้องเพิ่ม dependencies สองตัวหลักใน pubspec.yaml:

1. sqflite: ให้ API สำหรับการเปิดฐานข้อมูล, รันคำสั่ง SQL ดิบ (raw SQL), และดำเนินการ CRUD แบบสะดวก [3].
2. path_provider: ช่วยให้เราค้นหาพาธไดเร็กทอรีมาตรฐานบนระบบไฟล์ของอุปกรณ์ (เช่น ไดเร็กทอรีสำหรับเก็บข้อมูลแอปพลิเคชัน) ซึ่งเราสามารถจัดเก็บไฟล์ฐานข้อมูล .db ของเราได้อย่างปลอดภัย [4].

ขั้นตอนการเพิ่ม Dependencies:

1. เปิด pubspec.yaml.
2. ใต้ส่วน dependencies:, เพิ่ม sqflite และ path_provider. ตรวจสอบเวอร์ชันล่าสุดบน pub.dev.

```
dependencies:
  flutter:
    sdk: flutter
  path: ^1.9.1          # สำหรับ join() ต่อชื่อไฟล์เข้ากับโฟลเดอร์
  provider: ^6.1.5+1    # จากบทที่ 10
  http: ^1.6.0          # จากบทที่ 12
  sqflite: ^2.4.4       # เพิ่ม sqflite (ตรวจสอบเวอร์ชันล่าสุด)
  path_provider: ^2.1.6 # เพิ่ม path_provider (ตรวจสอบเวอร์ชันล่าสุด)
  # ... dependencies อื่นๆ ...
```

3. บันทึกไฟล์และรัน flutter pub get.

13.4 รูปแบบ Database Helper และ Singleton

การโต้ตอบกับฐานข้อมูลเกี่ยวข้องกับการเปิดการเชื่อมต่อ, การรันคำสั่ง, และการปิดการเชื่อมต่อ. เพื่อหลีกเลี่ยงการเปิดและปิดฐานข้อมูลซ้ำๆ และเพื่อให้มีจุดเข้าถึงฐานข้อมูลเพียงจุดเดียวในแอปพลิเคชัน รูปแบบการออกแบบทั่วไปคือการสร้าง Database Helper Class โดยใช้ Singleton Pattern.

- Database Helper Class: คลาสที่รวมตรรกะทั้งหมดที่เกี่ยวข้องกับฐานข้อมูลไว้ด้วยกัน (การเปิด, การสร้างตาราง, CRUD operations).
- Singleton Pattern: รูปแบบการออกแบบที่จำกัดการสร้างอินสแตนซ์ของคลาสให้มีเพียงอินสแตนซ์เดียว และให้จุดเข้าถึงส่วนกลางไปยังอินสแตนซ์นั้น. ทำให้มั่นใจได้ว่าเราทำงานกับออบเจกต์ฐานข้อมูลเดียวกันเสมอทั่วทั้งแอป.

ภาพที่ช่วยให้เข้าใจ Singleton คือห้องนิรภัยที่มีประตูบานเดียว ไฟล์ .db คือห้องนิรภัยนั้น ถ้าแอปมีหน้าจอเพิ่มรายการ หน้าจอสรุปยอด และหน้าจอตั้งค่า แล้วทุกหน้าจอต่างทำกุญแจของตัวเองเพื่อไขประตูเข้าไปเมื่อไรก็ได้ การดูแลห้องจะไม่มีใครรับผิดชอบจริง Singleton คือการตั้งกฎว่าทั้งบริษัทมีกุญแจดอกเดียว ถืออยู่ในมือผู้จัดการคนเดียวคือ DatabaseHelper.instance ใครอยากเก็บของหรือเบิกข้อมูลต้องยื่นเรื่องผ่านผู้จัดการคนนี้ ทุกคำสั่งจึงวิ่งผ่านท่อเดียวกันอย่างเป็นระเบียบ

เหตุผลทางเทคนิคที่แท้จริงมีสามข้อ ข้อแรกการเปิดไฟล์ฐานข้อมูลมีต้นทุน ควรเปิดครั้งเดียวแล้วใช้ต่อไปตลอดอายุของแอป ข้อสองต้องมีผู้รับผิดชอบการสร้างตารางและการเปลี่ยนโครงสร้างใน onCreate และ onUpgrade เพียงจุดเดียว และข้อสาม SQLite ยอมให้มีผู้เขียนได้ทีละรายเท่านั้น ถ้ามีการเชื่อมต่อหลายเส้นพยายามเขียนพร้อมกัน เส้นที่มาทีหลังต้องรอ และถ้ารอนานเกินไปจะได้ข้อผิดพลาด database is locked การรวมทุกคำสั่งไว้ที่การเชื่อมต่อเดียวทำให้คำสั่งเข้าคิวกันเอง

อย่างเป็นระเบียบ (แพ็กเกจ sqflite เองก็คืนการเชื่อมต่อตัวเดิมให้เมื่อเปิดไฟล์เดิมซ้ำ แต่การพึ่งพฤติกรรมนี้โดยไม่มีผู้ดูแลกลางทำให้โค้ดสร้างตารางกระจายอยู่หลายที่)

13.4.1 การเขียนโค้ด: DatabaseHelper และการเริ่มต้นฐานข้อมูล

เราจะใช้รูปแบบการออกแบบที่เรียกว่า Singleton เพื่อให้แน่ใจว่าเราจะมีอินสแตนซ์ของ DatabaseHelper เพียงอินสแตนซ์เดียวตลอดทั้งแอป ซึ่งป้องกันการเปิดการเชื่อมต่อฐานข้อมูลหลายครั้งโดยไม่จำเป็น

1. คำแนะนำ: ในโฟลเดอร์ lib, สร้างโฟลเดอร์ใหม่ชื่อ helpers
2. คำแนะนำ: ภายใน lib/helpers, สร้างไฟล์ใหม่ชื่อ database_helper.dart
3. คำแนะนำ: เพิ่มโค้ดส่วนเริ่มต้นของคลาส DatabaseHelper

```dart
// lib/helpers/database_helper.dart
import 'dart:io';
import 'package:path/path.dart';
import 'package:path_provider/path_provider.dart';
import 'package:sqflite/sqflite.dart';

class DatabaseHelper {
  static const _databaseName = "MyNotes.db";
  static const _databaseVersion = 1;

  // ทำให้คลาสนี้เป็น Singleton
  DatabaseHelper._privateConstructor();
  static final DatabaseHelper instance = DatabaseHelper._privateConstructor();

  // มี reference ไปยังฐานข้อมูลเพียงหนึ่งเดียว
  static Database? _database;
  Future<Database> get database async {
    if (_database != null) return _database!;
    // หาก _database เป็น null จะทำการ initialize ให้
    _database = await _initDatabase();
    return _database!;
  }

  // เมธอดสำหรับเปิดฐานข้อมูล (หรือสร้างขึ้นใหม่ถ้ายังไม่มี)
  Future<Database> _initDatabase() async {
    Directory documentsDirectory = await getApplicationDocumentsDirectory();
    String path = join(documentsDirectory.path, _databaseName);
    print('Database path: $path'); // แสดง path ของฐานข้อมูลใน console
    return await openDatabase(
      path,
      version: _databaseVersion,
```

```
      onCreate: _onCreate, // จะถูกเรียกเมื่อฐานข้อมูลถูกสร้างขึ้นครั้งแรก
    );
  }

  // เมธอดสำหรับสร้างตาราง (จะเพิ่มโค้ดในภายหลัง)
  Future<void> _onCreate(Database db, int version) async {
    // ในขั้นตอนนี้ เราจะยังไม่สร้างตาราง
    print('onCreate ถูกเรียก: กำลังจะสร้างตาราง...');
  }
}
```

คำอธิบาย:

1. _databaseName, _databaseVersion: ค่าคงที่สำหรับชื่อไฟล์ฐานข้อมูลและเวอร์ชันปัจจุบันของ schema.
2. _privateConstructor, instance: ทำให้คลาสเป็น Singleton. คุณจะเข้าถึง helper ผ่าน DatabaseHelper.instance.
3. _database: ตัวแปร static ที่เก็บการอ้างอิงไปยัง Database ที่เปิดอยู่ (เป็น nullable ในตอนแรก).
4. get database: Getter แบบ asynchronous. ตรวจสอบว่า _database ถูก initialize แล้วหรือยัง. ถ้ายัง ให้เรียก _initDatabase() เพื่อเปิดมัน แล้วจึงส่งคืนอินสแตนซ์ Database.
5. _initDatabase():
    - ใช้ getApplicationDocumentsDirectory() จาก path_provider เพื่อหาไดเร็กทอรีที่เหมาะสมสำหรับเก็บข้อมูลแอป.
    - ใช้ join() จากแพ็กเกจ path เพื่อสร้างพาธเต็มไปยังไฟล์ฐานข้อมูล.
    - เรียก openDatabase() จาก sqflite. ฟังก์ชันนี้จะเปิดฐานข้อมูลที่มีอยู่ หรือสร้างไฟล์ใหม่หากยังไม่มี.
    - version: ระบุเวอร์ชัน schema ปัจจุบัน.
    - onCreate: Callback ที่จะถูกเรียก *เฉพาะ* ครั้งแรกที่ไฟล์ฐานข้อมูลถูกสร้างขึ้น. เป็นที่ที่เหมาะสำหรับรันคำสั่ง CREATE TABLE.

_onCreate(): ฟังก์ชันที่คุณกำหนดเพื่อสร้างตารางเริ่มต้นเมื่อฐานข้อมูลถูกสร้างขึ้น. รับออบเจกต์ Database (db) ที่คุณสามารถใช้รันคำสั่ง SQL ได้.

ระบุชนิดคืนค่าให้ครบทุกเมธอด สังเกตว่า _initDatabase() ประกาศเป็น Future<Database> ถ้าละไว้ Dart จะถือว่าคืนค่าเป็น dynamic ตัววิเคราะห์โค้ดจึงไม่เตือนเลยหากเราคืนค่าผิดชนิด และข้อผิดพลาดจะไปโผล่ตอนรันแทน

ตำแหน่งของไฟล์ฐานข้อมูล ตัวอย่างนี้ใช้ getApplicationDocumentsDirectory() ของ path_provider อีกทางหนึ่งคือ getDatabasesPath() ของ sqflite เอง ซึ่งคืนโฟลเดอร์ฐานข้อมูลมาตรฐานของ Android และโฟลเดอร์ documents บน iOS ปฏิบัติการท้ายบทใช้วิธีหลัง ทั้งสองวิธีใช้ได้ แต่เมื่อเลือกแล้วห้ามเปลี่ยนหลังแอปถูกใช้งานจริง เพราะ openDatabase จะไปสร้างไฟล์ว่างใหม่ที่ตำแหน่งใหม่ และผู้ใช้จะเห็นข้อมูลเดิมหายไปทั้งหมด

ส่วนเหตุผลที่ต้องให้ path_provider หรือ getDatabasesPath() หาโฟลเดอร์ให้ แทนที่จะกำหนดเองแบบบนคอมพิวเตอร์ เช่นไดรฟ์ C หรือโฟลเดอร์ Documents ของผู้ใช้ คือระบบปฏิบัติการมือถือทั้ง Android และ iOS ใช้กลไกความปลอดภัยที่เรียกว่า sandboxing แอปแต่ละตัวถูกกั้นให้อยู่ในพื้นที่ของตัวเอง เขียนไฟล์นอกพื้นที่นั้นไม่ได้ และโครงสร้างโฟลเดอร์ของสองระบบก็ไม่เหมือนกัน ฟังก์ชันเหล่านี้จึงทำหน้าที่ถามระบบปฏิบัติการให้ว่า บนเครื่องนี้โฟลเดอร์ใดคือบ้านพักที่ปลอดภัยของแอป ที่ไฟล์ .db จะไม่ถูกลบทิ้งเมื่อระบบเคลียร์พื้นที่ ซึ่งต่างจากโฟลเดอร์ cache ที่ระบบลบได้ทุกเมื่อ

13.4.2 การทดสอบ: การเชื่อมต่อฐานข้อมูล

ตอนนี้เราจะสร้าง UI ง่ายๆ เพื่อทดสอบว่าคลาส DatabaseHelper ของเราสามารถสร้างไฟล์ฐานข้อมูลได้สำเร็จหรือไม่

1. คำแนะนำ: ไปที่ lib/main.dart และแทนที่โค้ดทั้งหมดด้วยโค้ดด้านล่าง

```dart
// lib/main.dart
import 'package:flutter/material.dart';
import 'helpers/database_helper.dart';

void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
  const MyApp({super.key});
  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      title: 'SQLite Lab',
      home: MyHomePage(),
    );
  }
}

class MyHomePage extends StatelessWidget {
  const MyHomePage({super.key});

  void _testInitDatabase() async {
    // การเข้าถึง getter 'database' จะเป็นการกระตุ้น _initDatabase()
    final db = await DatabaseHelper.instance.database;
    print('Database initialized: ${db.path}');
    // isopen เป็น property ที่บอกสถานะการเชื่อมต่อ
    print('Database is open: ${db.isOpen}');
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('SQLite Connection Test')),
      body: Center(
        child: ElevatedButton(
          onPressed: _testInitDatabase,
          child: const Text('Initialize & Test Database'),
        ),
      ),
    );
  }
}
```

2. คำแนะนำ: รันแอปพลิเคชันและกดปุ่ม "Initialize & Test Database"
3. ผลลัพธ์ที่คาดหวัง (Console): คุณควรจะเห็นข้อความใน Debug Console ที่แสดง path ของไฟล์ฐานข้อมูล และยืนยันว่าการเชื่อมต่อสำเร็จ ซึ่งเป็นการยืนยันว่า _initDatabase ทำงานถูกต้อง

```
I/flutter ( 1234): Database path:
/data/user/0/com.example.sqlite_lab/app_flutter/MyNotes.db
I/flutter ( 1234): onCreate ถูกเรียก: กำลังจะสร้างตาราง...
I/flutter ( 1234): Database initialized:
/data/user/0/com.example.sqlite_lab/app_flutter/MyNotes.db
I/flutter ( 1234): Database is open: true
```

13.5 การสร้างตาราง

ภายใน callback _onCreate (หรือ onUpgrade หากคุณกำลังอัปเดต schema), คุณจะใช้เมธอด db.execute() เพื่อรันคำสั่ง SQL CREATE TABLE [8].

ไวยากรณ์ CREATE TABLE พื้นฐาน:

```
CREATE TABLE table_name (
  column1_name DATATYPE column1_constraints,
  column2_name DATATYPE column2_constraints,
  ...
  table_constraints
);
```

ส่วนประกอบสำคัญ:

- table_name: ชื่อตารางที่คุณต้องการสร้าง (เช่น notes, tasks, products).
- column_name: ชื่อคอลัมน์ภายในตาราง.
- DATATYPE: ประเภทข้อมูลสำหรับคอลัมน์. ประเภทข้อมูลทั่วไปใน SQLite ที่แมปกับ Dart ได้ดีคือ:
    - INTEGER: สำหรับเก็บค่าจำนวนเต็ม (Dart int). มักใช้สำหรับ ID.
    - TEXT: สำหรับเก็บค่าสตริง (Dart String).
    - REAL: สำหรับเก็บค่าทศนิยม (Dart double).
    - BLOB: สำหรับเก็บข้อมูลไบนารี (Dart Uint8List).
    - NUMERIC: สามารถเก็บค่าได้หลากหลาย (ไม่แนะนำให้ใช้บ่อยเท่าประเภทอื่น).
    - (SQLite มี affinity type ที่ยืดหยุ่น แต่การใช้ประเภทเหล่านี้จะชัดเจนที่สุด).
- column_constraints: กฎเพิ่มเติมสำหรับคอลัมน์:
    - PRIMARY KEY: ระบุว่าคอลัมน์นี้เป็นคีย์หลัก (unique identifier) ของตาราง. มักใช้กับคอลัมน์ ID.
        - AUTOINCREMENT (ใช้กับ INTEGER PRIMARY KEY เท่านั้น): ทำให้ SQLite สร้างค่า ID ที่ไม่ซ้ำกันและเพิ่มขึ้นโดยอัตโนมัติเมื่อมีการแทรกแถวใหม่ (คุณไม่ต้องระบุ ID ตอน insert).

- NOT NULL: บังคับว่าคอลัมน์นี้ต้องมีค่า ไม่สามารถเป็น NULL ได้.
- UNIQUE: ทำให้แน่ใจว่าค่าทั้งหมดในคอลัมน์นี้ไม่ซ้ำกัน.
- DEFAULT value: กำหนดค่าเริ่มต้นหากไม่มีการระบุค่าตอน insert.

- table_constraints: (ทางเลือก) กฎที่ใช้กับทั้งตาราง (เช่น FOREIGN KEY).

13.5.1 การเขียนโค้ด: _onCreate

1. คำแนะนำ: กลับไปที่ lib/helpers/database_helper.dart และแก้ไขเมธอด _onCreate เพื่อเพิ่มคำสั่ง SQL สำหรับสร้างตาราง notes

```dart
// ภายในคลาส DatabaseHelper
Future _onCreate(Database db, int version) async {
  print('onCreate ถูกเรียก: กำลังจะสร้างตาราง notes...');
  await db.execute('''
        CREATE TABLE notes (
          id INTEGER PRIMARY KEY AUTOINCREMENT,
          title TEXT NOT NULL,
          content TEXT NOT NULL
        )
        ''');
  print('ตาราง notes ถูกสร้างแล้ว');
}
```

หมายเหตุ: เมธอด _onCreate จะทำงานเพียงครั้งเดียวเมื่อไฟล์ฐานข้อมูลถูกสร้างขึ้น ถ้าเราเคยรันโค้ดในหัวข้อ 13.4.2 ไปแล้ว ไฟล์ MyNotes.db มีอยู่แล้วแต่ยังไม่มีตาราง notes ระหว่างพัฒนา วิธีที่เร็วที่สุดในการบังคับให้ _onCreate ทำงานอีกครั้งคือถอนการติดตั้ง (uninstall) แอปออกจาก emulator หรืออุปกรณ์แล้วรันใหม่ แต่วิธีนี้ใช้ได้กับเครื่องของนักพัฒนาเท่านั้น เพราะการถอนการติดตั้งคือการลบข้อมูลทั้งหมดของผู้ใช้ทิ้ง เมื่อแอปถูกใช้งานจริงแล้วต้องเปลี่ยนโครงสร้างตารางด้วย onUpgrade ซึ่งอธิบายไว้ในหัวข้อ 13.8

13.6 การดำเนินการ CRUD

เมื่อฐานข้อมูลและตารางถูกสร้างขึ้นแล้ว ขั้นตอนต่อไปคือการเพิ่มเมธอดใน DatabaseHelper ของคุณเพื่อดำเนินการ CRUD (Create, Read, Update, Delete). เมธอดเหล่านี้มักจะรับและส่งคืน Map<String, dynamic>.

13.6.1 การเพิ่มข้อมูล (Create)

1. การเขียนโค้ดเพิ่มข้อมูล: เพิ่มเมธอด insert สำหรับการเพิ่มข้อมูลใหม่ลงในตาราง notes เมธอด db.insert() รับชื่อตารางและ Map ของค่าในแต่ละคอลัมน์ แล้วคืน id ของแถวที่สร้างขึ้น [9]

```dart
// ภายในคลาส DatabaseHelper
// Create/Insert
Future<int> insert(Map<String, dynamic> row) async {
  Database db = await instance.database;
```

```
  // db.insert จะคืนค่า id ของแถวที่ถูกเพิ่มล่าสุด
  return await db.insert('notes', row);
}
```

2. การทดสอบเพิ่มข้อมูล: กลับไปที่ lib/main.dart และปรับปรุง UI เพื่อให้สามารถป้อนข้อมูลและมีปุ่มสำหรับ "Insert"

```
// lib/main.dart (ส่วนของ MyHomePage)
class MyHomePage extends StatefulWidget {
  const MyHomePage({super.key});
  @override
  State<MyHomePage> createState() => _MyHomePageState();
}

class _MyHomePageState extends State<MyHomePage> {
  final dbHelper = DatabaseHelper.instance;
  final TextEditingController _titleController = TextEditingController();
  final TextEditingController _contentController = TextEditingController();

  @override
  void dispose() {
    // คืนหน่วยความจำของ controller เมื่อหน้าจอถูกทำลาย (บทที่ 8)
    _titleController.dispose();
    _contentController.dispose();
    super.dispose();
  }

  void _insert() async {
    // สร้าง Map ของข้อมูลที่จะเพิ่ม
    Map<String, dynamic> row = {
      'title': _titleController.text,
      'content': _contentController.text,
    };
    // เรียกใช้เมธอด insert และรับ id ที่ถูกสร้างขึ้นกลับมา
    final id = await dbHelper.insert(row);
    print('Inserted row id: $id');
    // เคลียร์ TextField หลังจากเพิ่มข้อมูล
    _titleController.clear();
    _contentController.clear();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('SQLite C.R.U.D. Lab')),
      body: Padding(
        padding: const EdgeInsets.all(16.0),
        child: Column(
```

```
          children: [
            TextField(
              controller: _titleController,
              decoration: const InputDecoration(labelText: 'Title'),
            ),
            TextField(
              controller: _contentController,
              decoration: const InputDecoration(labelText: 'Content'),
            ),
            const SizedBox(height: 20),
            ElevatedButton(
              onPressed: _insert,
              child: const Text('Insert Note'),
            ),
            // พื้นที่สำหรับแสดงผล (จะเพิ่มในขั้นตอนถัดไป)
          ],
        ),
      ),
    );
  }
}
```

3. คำแนะนำ: ถอนการติดตั้งแอป แล้วรันใหม่อีกครั้ง เพื่อให้ _onCreate ที่มีคำสั่ง CREATE TABLE ทำงาน
4. คำแนะนำ: ป้อน "First Note" ในช่อง Title และ "This is the content of my first note." ในช่อง Content แล้วกดปุ่ม "Insert Note"
5. ผลลัพธ์ที่คาดหวัง (Console): คุณจะเห็นข้อความที่แสดง id ของแถวที่ถูกเพิ่ม ซึ่งโดยปกติจะเริ่มจาก 1

```
I/flutter ( 5678): Inserted row id: 1
```

13.6.2 การอ่านและแสดงผลข้อมูล (Read)

ใช้เมธอด db.query() เพื่อดึงข้อมูลแถวจากตาราง [10]. มีความยืดหยุ่นสูง:

- table: ชื่อตาราง.
- columns: (ทางเลือก) List<String> ของคอลัมน์ที่จะดึงข้อมูล (ถ้า null จะดึงทุกคอลัมน์ *).
- where: (ทางเลือก) เงื่อนไข WHERE ของ SQL (สตริง). ใช้ ? เป็น placeholder สำหรับ arguments.
- whereArgs: (ทางเลือก) List<dynamic> ของ arguments ที่จะแทนที่ ? ใน where (ป้องกัน SQL Injection).
- groupBy, having, orderBy: (ทางเลือก) สำหรับการจัดกลุ่ม, การกรองหลังจัดกลุ่ม, และการเรียงลำดับผลลัพธ์.
- limit, offset: (ทางเลือก) สำหรับการจำกัดจำนวนผลลัพธ์และการข้ามแถว (pagination).

query() ส่งคืน Future<List<Map<String, dynamic>>>. แต่ละ Map ในลิสต์แทนหนึ่งแถวจากฐานข้อมูล.

ทำไมต้องใช้ ? กับ whereArgs แทนการต่อข้อความ สมมติว่าเราทำช่องค้นหาโน้ตจากชื่อ แล้วเขียนเงื่อนไขด้วยการนำข้อความที่ผู้ใช้พิมพ์มาต่อเข้ากับคำสั่ง SQL ตรง ๆ ถ้าผู้ใช้พิมพ์ข้อความที่มีเครื่องหมาย ' ปนมา คำสั่งจะพังทันที และถ้าพิมพ์อย่างจงใจ ข้อความนั้นจะกลายเป็นส่วนหนึ่งของคำสั่ง SQL เอง นี่คือช่องโหว่ที่เรียกว่า

SQL Injection

```
// อันตราย: ข้อความของผู้ใช้กลายเป็นส่วนหนึ่งของคำสั่ง SQL
// ถ้าผู้ใช้พิมพ์  x' OR '1'='1  เงื่อนไขจะเป็นจริงทุกแถว
final rows = await db.rawQuery(
  "SELECT * FROM notes WHERE title = '$keyword'",
);

// ปลอดภัย: ? เป็นช่องว่างให้ sqflite ใส่ค่าให้เอง ค่านั้นเป็นข้อมูลเสมอ ไม่ใช่คำสั่ง
final safeRows = await db.query(
  'notes',
  where: 'title = ?',
  whereArgs: [keyword],
);
```

กฎที่จำง่ายคือ ข้อความที่มาจากผู้ใช้หรือจากเครือข่ายต้องเข้าไปทาง whereArgs เท่านั้น ห้ามใช้การแทรกค่าด้วย $ ลงในสตริงของคำสั่ง SQL ไม่ว่าจะใช้ query() หรือ rawQuery() ก็ตาม เพราะ rawQuery() ก็รับรายการค่าเป็นพารามิเตอร์ที่สองได้เช่นกัน

หลายแหล่งอธิบายว่าการใช้ ? ทำให้ระบบทำความสะอาด (sanitize) ข้อมูลให้ก่อน ซึ่งไม่ตรงกับกลไกจริงนัก สิ่งที่เกิดขึ้นคือ SQLite แปลงข้อความคำสั่งที่มี ? ให้เป็นคำสั่งที่พร้อมทำงานก่อน โดยยังไม่รู้ค่าจริงเลย แล้วจึงผูกค่าจาก whereArgs เข้าไปในช่องเหล่านั้นภายหลังในฐานะข้อมูล ค่าที่ผูกเข้าไปจึงไม่มีทางถูกอ่านเป็นส่วนหนึ่งของคำสั่ง SQL ได้ ไม่ว่าจะมีเครื่องหมายคำพูดหรือคำว่า DROP TABLE อยู่ข้างในก็ตาม ข้อมูลไม่ได้ถูกแก้ไขหรือกรองแต่อย่างใด เครื่องหมาย ' ที่ผู้ใช้พิมพ์จะถูกเก็บลงฐานข้อมูลตามจริงทุกตัว

1. การเขียนโค้ดการอ่านข้อมูล: กลับไปที่ database_helper.dart และเพิ่มเมธอด queryAllRows

```
// ภายในคลาส DatabaseHelper
// Read/Query
Future<List<Map<String, dynamic>>> queryAllRows() async {
  Database db = await instance.database;
  // db.query จะคืนค่าเป็น List ของ Map
  // orderBy ช่วยให้เราเรียงลำดับผลลัพธ์ (ในที่นี้คือเรียงตาม id จากมากไปน้อย)
  return await db.query('notes', orderBy: 'id DESC');
}
```

2. การทดสอบแสดงผลข้อมูล: กลับไปที่ lib/main.dart และปรับปรุง _MyHomePageState เพื่อดึงและแสดงข้อมูล

```
// ภายในคลาส _MyHomePageState
List<Map<String, dynamic>> _allNotes = [];

@override
void initState() {
  super.initState();
  _refreshNotes(); // เรียกใช้เมื่อ state เริ่มต้น
}

void _refreshNotes() async {
```

```
  final data = await dbHelper.queryAllRows();
  // ระหว่างรอ ผู้ใช้อาจออกจากหน้าจอไปแล้ว (บทที่ 12 หัวข้อ 12.2)
  if (!mounted) return;
  setState(() { // ใช้ setState เพื่ออัปเดต UI
    _allNotes = data;
  });
}

void _insert() async {
  // ... (โค้ดเดิม) ...
  _refreshNotes(); // เรียก refresh หลังจากเพิ่มข้อมูล
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: const Text('SQLite C.R.U.D. Lab')),
    body: Padding(
      padding: const EdgeInsets.all(16.0),
      child: Column(
        children: [
          // ... (ส่วนของ TextField และปุ่ม Insert) ...
          const SizedBox(height: 20),
          // ใช้ Expanded เพื่อให้ ListView ใช้พื้นที่ที่เหลือทั้งหมด
          Expanded(
            child: ListView.builder(
              itemCount: _allNotes.length,
              itemBuilder: (context, index) {
                final note = _allNotes[index];
                return Card(
                  margin: const EdgeInsets.symmetric(vertical: 5),
                  child: ListTile(
                    title: Text(note['title']),
                    subtitle: Text(note['content']),
                    // trailing จะเพิ่มในขั้นตอนถัดไป
                  ),
                );
              },
            ),
          ),
        ],
      ),
    ),
  );
}
```

3. คำแนะนำ: รันแอปพลิเคชัน (Hot Restart)

4. ผลลัพธ์ที่คาดหวัง (UI): คุณควรจะเห็น "First Note" ที่คุณเพิ่มไว้ก่อนหน้านี้ปรากฏขึ้นมาเป็น Card บนหน้าจอ ลองเพิ่มโน้ตใหม่ๆ แล้วคุณจะเห็นว่าลิสต์อัปเดตทันที

13.6.3 การแก้ไขข้อมูล (Update)

ใช้เมธอด db.update() เพื่อแก้ไขแถวที่มีอยู่ [11].

- table: ชื่อตาราง.
- values: Map<String, dynamic> ที่มีค่าคอลัมน์ใหม่ที่จะอัปเดต.
- where: (ทางเลือก) เงื่อนไข WHERE เพื่อระบุแถวที่จะอัปเดต (สำคัญมาก! ถ้าไม่ระบุจะอัปเดตทุกแถว).
- whereArgs: (ทางเลือก) Arguments สำหรับ where.
- conflictAlgorithm: (ทางเลือก) การจัดการข้อขัดแย้ง.

update() ส่งคืน Future<int> ซึ่งเป็นจำนวนแถวที่ได้รับผลกระทบ.

1. การเขียนโค้ดแก้ไขข้อมูล: กลับไปที่ database_helper.dart และเพิ่มเมธอด update และ delete

```
// ภายในคลาส DatabaseHelper
// Update
Future<int> update(Map<String, dynamic> row) async {
  Database db = await instance.database;
  int id = row['id'];
  // db.update จะคืนค่าจำนวนแถวที่ได้รับผลกระทบ
  return await db.update('notes', row, where: 'id = ?', whereArgs: [id]);
}
```

2. การทดสอบแก้ไขข้อมูล: กลับไปที่ lib/main.dart และเพิ่มเมธอด _update, _delete และเพิ่มปุ่มใน ListTile

```
// ภายในคลาส _MyHomePageState

void _update(int id) async {
  // ข้อมูลใหม่ที่จะนำไปอัปเดต
  Map<String, dynamic> row = {
    'id': id,
    'title': 'Updated Title',
    'content': 'This content has been updated.',
  };
  final rowsAffected = await dbHelper.update(row);
  print('Updated $rowsAffected row(s)');
  _refreshNotes();
}

// ภายในเมธอด build, แก้ไข ListTile ใน ListView.builder
// ...
return Card(
```

```
  margin: const EdgeInsets.symmetric(vertical: 5),
  child: ListTile(
    title: Text(note['title']),
    subtitle: Text(note['content']),
    trailing: Row(
      mainAxisSize: MainAxisSize.min, // ทำให้ Row ใช้พื้นที่น้อยที่สุด
      children: [
        IconButton(
          icon: const Icon(Icons.edit, color: Colors.blue),
          onPressed: () => _update(note['id']),
        ),
      ],
    ),
  ),
);
// ...
```

3. คำแนะนำ: รันแอปพลิเคชัน (Hot Restart)
4. ทดสอบการอัปเดต: กดไอคอนดินสอ (Edit) ที่โน้ตใดๆ คุณจะเห็นว่า title และ content ของโน้ตนั้นเปลี่ยนไปเป็นค่าใหม่ทันที
5. ทดสอบการคงอยู่ของข้อมูล (Persistence): ปิดแอปพลิเคชันแล้วเปิดใหม่ ข้อมูลหลังจากที่คุณได้แก้ไขไปแล้ว ควรจะยังคงอยู่เหมือนเดิม

13.6.4 การลบข้อมูล (Delete)

ใช้เมธอด db.delete() เพื่อลบแถวออกจากตาราง [12].

- table: ชื่อตาราง.
- where: (ทางเลือก) เงื่อนไข WHERE เพื่อระบุแถวที่จะลบ (สำคัญมาก! ถ้าไม่ระบุจะลบทุกแถว).
- whereArgs: (ทางเลือก) Arguments สำหรับ where.

delete() ส่งคืน Future<int> ซึ่งเป็นจำนวนแถวที่ถูกลบ.

กฎเหล็กของ update() และ delete() ฐานข้อมูลเป็นเครื่องจักรที่ซื่อตรงมาก มันทำตามคำสั่งตรงตัวอักษรโดยไม่ถามซ้ำ ถ้าเราสั่งลบโดยระบุแค่ชื่อตารางแต่ลืมใส่ where ฐานข้อมูลจะเข้าใจว่าเราต้องการลบทุกแถวในตารางนั้น การกดลบรายการผิดเพียงรายการเดียวจึงอาจกลายเป็นการลบประวัติการเงินทั้งหมดของผู้ใช้ภายในเสี้ยววินาที และ update() ที่ไม่มี where ก็จะเขียนทับทุกแถวด้วยค่าเดียวกันในลักษณะเดียวกัน ทุกครั้งที่เรียกสองเมธอดนี้ให้ตรวจก่อนเสมอว่ามี where และ whereArgs ครบ

1. การเขียนโค้ดลบข้อมูล: กลับไปที่ database_helper.dart และเพิ่มเมธอด delete

```
// ภายในคลาส DatabaseHelper
// Delete
Future<int> delete(int id) async {
  Database db = await instance.database;
```

```
  // db.delete จะคืนค่าจำนวนแถวที่ถูกลบ
  return await db.delete('notes', where: 'id = ?', whereArgs: [id]);
}
```

2. การทดสอบลบข้อมูล: กลับไปที่ lib/main.dart และเพิ่มเมธอด _delete และเพิ่มปุ่มใน ListTile

```
// ภายในคลาส _MyHomePageState

void _update(int id) async { ... }

void _delete(int id) async {
  final rowsDeleted = await dbHelper.delete(id);
  print('Deleted $rowsDeleted row(s): row $id');
  _refreshNotes();
}

// ภายในเมธอด build, แก้ไข ListTile ใน ListView.builder
// ...
return Card(
  margin: const EdgeInsets.symmetric(vertical: 5),
  child: ListTile(
    title: Text(note['title']),
    subtitle: Text(note['content']),
    trailing: Row(
      mainAxisSize: MainAxisSize.min, // ทำให้ Row ใช้พื้นที่น้อยที่สุด
      children: [
        IconButton(
          icon: const Icon(Icons.edit, color: Colors.blue),
          onPressed: () => _update(note['id']),
        ),
        IconButton(//<-- เพิ่มปุ่ม delete ต่อจากปุ่ม update ที่นี่
          icon: const Icon(Icons.delete, color: Colors.red),
          onPressed: () => _delete(note['id']),
        ),
      ],
    ),
  ),
);
// ...
```

3. คำแนะนำ: รันแอปพลิเคชัน (Hot Restart)
4. ทดสอบการลบ: กดไอคอนถังขยะ (Delete) ที่โน้ตใดๆ คุณจะเห็นว่าโน้ตนั้นหายไปจากลิสต์
5. ทดสอบการคงอยู่ของข้อมูล (Persistence): ปิดแอปพลิเคชันแล้วเปิดใหม่ ข้อมูลที่เหลืออยู่ (หลังจากที่คุณได้ลบไปแล้ว) ควรจะยังคงอยู่เหมือนเดิม

ยินดีด้วย! คุณได้สร้างและทดสอบการทำงานของฐานข้อมูล SQLite ใน Flutter ครบทั้ง 4 กระบวนการหลัก (Create, Read, Update, Delete) แล้ว ตอนนี้คุณมีพื้นฐานที่แข็งแกร่งและมีคลาส DatabaseHelper ที่พร้อมจะนำไปประยุกต์ใช้ในโปรเจกต์ Task Manager ในบทถัดไป

13.7 การแปลงข้อมูลระหว่าง Map กับ Model Class

โค้ดใน 13.6 ส่ง Map<String, dynamic> ไปมาระหว่างหน้าจอกับฐานข้อมูลโดยตรง วิธีนี้เข้าใจง่ายสำหรับการเริ่มต้น แต่มีปัญหาเดียวกับที่บทที่ 12 พบกับ JSON คือถ้าพิมพ์ชื่อคีย์ผิดแม้แต่ตัวเดียว เช่น note['tilte'] ตัววิเคราะห์โค้ดจะไม่เตือนเลย และแอปจะพังตอนรัน วิธีแก้ก็เป็นแบบเดียวกัน คือสร้าง Model Class ที่มีเมธอด toMap() สำหรับแปลงอ็อบเจกต์เป็นแถวก่อนบันทึก และ factory constructor ชื่อ fromMap() สำหรับแปลงแถวที่อ่านได้กลับเป็นอ็อบเจกต์

ปัญหาที่อยู่เบื้องหลังเรื่องนี้มีชื่อเรียกในทางวิศวกรรมซอฟต์แวร์ว่า Object-Relational Impedance Mismatch หรือความไม่เข้ากันระหว่างสองโลก ฝั่ง Dart เป็นโลกเชิงวัตถุที่มีคลาส ฟิลด์ ชนิด String, double และ bool ส่วนฝั่ง SQL มีแต่ตาราง แถว คอลัมน์ และชนิดข้อมูลไม่กี่ชนิด สองโลกนี้พูดกันคนละภาษา จึงต้องมีล่ามคั่นกลาง

ลองนึกถึงการส่งตู้เฟอร์นิเจอร์ข้ามทะเล ตู้ทั้งใบใส่เรือไม่ได้ ต้องถอดเป็นชิ้น ๆ บรรจุลงกล่องที่มีป้ายกำกับทุกชิ้นก่อน นั่นคือหน้าที่ของ toMap() ที่ถอดอ็อบเจกต์ออกเป็นคู่ของชื่อคอลัมน์กับค่า ซึ่ง sqflite นำไปหยอดลงคอลัมน์ที่ชื่อตรงกันได้แม่นยำ เมื่อถึงปลายทางก็ต้องมีคู่มือประกอบกลับเป็นตู้ใบเดิม นั่นคือหน้าที่ของ fromMap() ที่รับแถวซึ่งอ่านขึ้นมาในรูป Map แล้วประกอบกลับเป็นอ็อบเจกต์ที่หน้าจอใช้งานต่อได้ทันที

```dart
// lib/models/note.dart
class Note {
  final int? id; // null เมื่อยังไม่เคยบันทึก ฐานข้อมูลเป็นผู้กำหนดให้ตอน insert
  final String title;
  final String content;

  const Note({this.id, required this.title, required this.content});

  Map<String, Object?> toMap() {
    return {
      if (id != null) 'id': id, // ใส่ id เฉพาะเมื่อมีค่า
      'title': title,
      'content': content,
    };
  }

  factory Note.fromMap(Map<String, Object?> map) {
    return Note(
      id: map['id'] as int,
      title: map['title'] as String,
      content: map['content'] as String,
    );
  }
}
```

จุดที่ต้องสังเกตคือบรรทัด if (id != null) ใน toMap() ถ้าเขียนแบบตรงไปตรงมาว่า 'id': id ทุกครั้ง อ็อบเจกต์ที่ยังไม่มี id จะส่งค่า null ลงไปในคอลัมน์ id ตอน insert ซึ่ง SQLite ยอมรับและสร้างเลขใหม่ให้ แต่ถ้านำอ็อบเจกต์แบบเดียวกันไปใช้กับ update() คำสั่งจะกลายเป็นการสั่งเปลี่ยนคีย์หลักของแถวให้เป็น null และ SQLite จะปฏิเสธด้วยข้อผิดพลาด datatype mismatch การใส่ id เฉพาะเมื่อมีค่าจึงทำให้ toMap() ตัวเดียวใช้ได้กับทั้งสองคำสั่ง

เมื่อมี Model Class แล้ว เมธอดใน DatabaseHelper ก็รับและคืนเป็น Note แทน Map หน้าจอจึงเขียนว่า note.title ได้ และถ้าพิมพ์ผิดจะรู้ตั้งแต่ตอนเขียนโค้ด

```dart
// ภายในคลาส DatabaseHelper
Future<int> insertNote(Note note) async {
  final db = await database;
  return db.insert('notes', note.toMap());
}

Future<List<Note>> queryAllNotes() async {
  final db = await database;
  final rows = await db.query('notes', orderBy: 'id DESC');
  return rows.map(Note.fromMap).toList();
}

Future<int> updateNote(Note note) async {
  final db = await database;
  return db.update('notes', note.toMap(), where: 'id = ?', whereArgs: [note.id]);
}
```

ข้อควรรู้อีกข้อคือ Map ที่ได้จาก query() เป็นแบบอ่านอย่างเดียว ถ้าพยายามแก้ค่าในนั้นโดยตรงจะเกิดข้อผิดพลาดตอนรัน การแปลงเป็นอ็อบเจกต์ทันทีที่อ่านขึ้นมาจึงตัดปัญหานี้ไปด้วย

สำหรับข้อมูลชนิดที่ SQLite ไม่มี ให้ตกลงวิธีแปลงไว้ที่ toMap() และ fromMap() เพียงจุดเดียว แนวทางที่ใช้กันทั่วไปมีดังนี้

- bool เก็บเป็น INTEGER ค่า 1 หรือ 0 ตอนบันทึกเขียน isDone ? 1 : 0 และตอนอ่านเขียน map['is_done'] == 1
- DateTime เก็บเป็น TEXT ด้วย toIso8601String() ซึ่งอ่านง่ายเมื่อเปิดดูไฟล์และเรียงตามตัวอักษรได้ถูกลำดับเวลา หรือเก็บเป็น INTEGER ด้วย millisecondsSinceEpoch ซึ่งเปรียบเทียบได้เร็วกว่า เลือกแบบใดแบบหนึ่งแล้วใช้ให้เหมือนกันทั้งแอป
- enum เก็บเป็น TEXT ด้วย .name และอ่านกลับด้วย values.byName() เช่น TransactionType.values.byName(map['type'] as String) อย่าเก็บเป็นลำดับตัวเลข (.index) เพราะถ้าวันหนึ่งเพิ่มค่าใหม่ไว้กลาง enum ข้อมูลเดิมทุกแถวจะชี้ไปผิดค่า
- double ที่อ่านจากคอลัมน์ REAL ให้แปลงด้วย (map['amount'] as num).toDouble() เพื่อรองรับกรณีที่แถวเก่าถูกบันทึกเป็นจำนวนเต็ม

13.8 การเปลี่ยนโครงสร้างตารางด้วย onUpgrade

สมมติว่าแอปจดบันทึกของเราถูกติดตั้งไปแล้วในเครื่องของผู้ใช้หลายพันคน ทุกคนมีโน้ตสะสมอยู่ในตาราง notes ที่มีสามคอลัมน์ วันหนึ่งเราต้องการเพิ่มคอลัมน์ created_at เพื่อเก็บเวลาที่สร้างโน้ต การแก้คำสั่ง CREATE TABLE ใน _onCreate เพียงอย่างเดียวไม่มีผลกับผู้ใช้เหล่านั้นเลย เพราะ _onCreate ทำงานครั้งเดียวตอนไฟล์ถูกสร้าง และการขอให้ผู้ใช้ถอนการติดตั้งแอปก็คือการลบโน้ตทั้งหมดของเขาทิ้ง

ถ้าเปรียบ _onCreate เป็นการสร้างตึกครั้งแรกบนที่ดินเปล่า ซึ่งทำได้เพียงครั้งเดียวในชีวิตของไฟล์ onUpgrade ก็คือการต่อเติมตึกที่มีคนอาศัยอยู่แล้ว เราทุบตึกทิ้งแล้วสร้างใหม่ไม่ได้เพราะข้าวของของผู้อยู่อาศัยอยู่ข้างใน ต้องต่อเติมทีละส่วนโดยที่ของเดิมยังอยู่ครบ

sqflite แก้ปัญหานี้ด้วยหมายเลขเวอร์ชันของฐานข้อมูล ทุกครั้งที่เปิดไฟล์ openDatabase จะเทียบเลขเวอร์ชันที่บันทึกไว้ในไฟล์กับค่า version ที่เราส่งเข้าไป ถ้ายังไม่มีไฟล์จะเรียก onCreate ถ้าเลขในไฟล์น้อยกว่าจะเรียก onUpgrade พร้อมบอกว่าเลขเดิมคืออะไร [13] และเมื่อทำงานเสร็จจะบันทึกเลขใหม่ลงไฟล์ ทั้งสาม callback ทำงานอยู่ภายใน transaction ถ้าคำสั่งใดล้มเหลว ไฟล์จะกลับไปอยู่ในสภาพเดิมทั้งหมด

```dart
// ภายในคลาส DatabaseHelper
static const _databaseVersion = 2; // เดิมคือ 1 เพิ่มขึ้นทุกครั้งที่โครงสร้างเปลี่ยน
```

```dart
Future<Database> _initDatabase() async {
  final documentsDirectory = await getApplicationDocumentsDirectory();
  final path = join(documentsDirectory.path, _databaseName);
  return openDatabase(
    path,
    version: _databaseVersion,
    onCreate: _onCreate,   // ผู้ใช้ใหม่ ยังไม่มีไฟล์
    onUpgrade: _onUpgrade, // ผู้ใช้เดิม ไฟล์มีเลขเวอร์ชันต่ำกว่า
  );
}

// สร้างตารางในรูปแบบล่าสุดเสมอ ผู้ใช้ใหม่จึงไม่ต้องไล่ทีละขั้น
Future<void> _onCreate(Database db, int version) async {
  await db.execute('''
    CREATE TABLE notes (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      title TEXT NOT NULL,
      content TEXT NOT NULL,
      created_at TEXT
    )
  ''');
}

// ไล่ปรับจากเวอร์ชันที่อยู่ในเครื่องขึ้นมาทีละขั้น
Future<void> _onUpgrade(Database db, int oldVersion, int newVersion) async {
  if (oldVersion < 2) {
    await db.execute('ALTER TABLE notes ADD COLUMN created_at TEXT');
  }
  // เวอร์ชันถัดไปให้เพิ่ม if (oldVersion < 3) ต่อท้ายตรงนี้
  // ห้ามแก้ขั้นที่ปล่อยให้ผู้ใช้ไปแล้ว เพราะมีเครื่องที่ผ่านขั้นนั้นไปแล้วจริง
}
```

กติกาที่ต้องจำมีสี่ข้อ

- _onCreate ต้องสร้างโครงสร้างล่าสุดเสมอ และต้องได้ผลตรงกับเครื่องที่ไล่ onUpgrade มาจนถึงเวอร์ชันเดียวกัน ถ้าสองทางนี้ได้ตารางไม่เหมือนกัน บั๊กจะเกิดเฉพาะกับผู้ใช้กลุ่มใดกลุ่มหนึ่ง ซึ่งหาสาเหตุยากมาก
- ใช้ if (oldVersion < N) เรียงต่อกัน ไม่ใช้ == หรือ switch เพราะผู้ใช้บางคนไม่ได้อัปเดตแอปทุกรุ่น เครื่องที่ข้ามจากเวอร์ชัน 1 ไปเวอร์ชัน 3 ต้องผ่านทั้งขั้นที่ 2 และขั้นที่ 3 ตามลำดับ ตัวอย่าง migration ในเอกสารของ sqflite เขียนเป็น if (oldVersion == 1) [13] ซึ่งใช้ได้เพราะตัวอย่างนั้นมีเพียงสองเวอร์ชัน แต่ถ้าแอปมีเวอร์ชัน 3 แล้วเขียนขั้นถัดไปเป็น if (oldVersion == 2) เครื่องที่ข้ามจากเวอร์ชัน 1 จะทำเฉพาะขั้นแรก แล้วไม่ได้คอลัมน์ของเวอร์ชัน 3 เลย เงื่อนไข < ครอบคลุมทุกกรณีและยังให้ผลเหมือน == 1 เมื่อมีเพียงสองเวอร์ชัน
- คอลัมน์ที่เพิ่มด้วย ALTER TABLE จะเป็น NULL ในแถวเดิมทั้งหมด คอลัมน์ใหม่จึงประกาศ NOT NULL โดยไม่มี DEFAULT ไม่ได้ [14] และ fromMap() ต้องรับมือกับค่า null ในคอลัมน์นั้นเสมอ เช่น map['created_at'] == null ? null : DateTime.parse(map['created_at'] as String)
- การลบหรือเปลี่ยนชื่อคอลัมน์ทำได้จำกัด ความสามารถของ ALTER TABLE ขึ้นกับรุ่นของ SQLite ที่มากับระบบปฏิบัติการ และอุปกรณ์รุ่นเก่ามีคำสั่งให้ใช้น้อยกว่า วิธีที่ใช้ได้กับทุกรุ่นคือสร้างตารางใหม่ด้วยโครงสร้างที่ต้องการ คัดลอกข้อมูลด้วย INSERT INTO ... SELECT ลบตารางเดิม แล้วเปลี่ยนชื่อตารางใหม่

ทั้งหมดนี้ทำภายใน onUpgrade ซึ่งอยู่ใน transaction อยู่แล้ว

วิธีทดสอบคือติดตั้งแอปรุ่นที่ใช้เวอร์ชัน 1 แล้วเพิ่มโน้ตไว้สองสามรายการ จากนั้นแก้โค้ดเป็นเวอร์ชัน 2 แล้วรันทับโดยไม่ถอนการติดตั้ง โน้ตเดิมต้องยังอยู่ครบและมี created_at เป็น null ส่วนโน้ตที่สร้างใหม่ต้องมีค่า การทดสอบเฉพาะกรณีติดตั้งใหม่จะไม่มีวันเจอบั๊กของ onUpgrade เพราะเส้นทางนั้นไม่เคยถูกเรียกเลย

13.9 ทำหลายคำสั่งให้สำเร็จพร้อมกันด้วย transaction และ batch

หัวข้อ 13.2 บอกว่า SQLite เป็นไปตามหลัก ACID แต่ตัวอย่างทั้งหมดที่ผ่านมาสั่งทีละคำสั่ง หลักข้อ Atomicity จะมีความหมายจริงก็ต่อเมื่อเราต้องทำหลายคำสั่งที่ต้องสำเร็จไปด้วยกันหรือไม่สำเร็จเลย ตัวอย่างที่พบบ่อยคือการนำข้อมูลชุดใหม่ทั้งชุดมาแทนชุดเดิม ถ้าลบข้อมูลเดิมเสร็จแล้วแอปถูกปิดกลางทางก่อนเพิ่มชุดใหม่เสร็จ ผู้ใช้จะเหลือข้อมูลครึ่งเดียวหรือไม่เหลือเลย

ตัวอย่างคลาสสิกคือการบันทึกการโอนเงินระหว่างสองบัญชี ซึ่งต้องทำสองขั้นตอน คือหักเงินจากบัญชี A แล้วเพิ่มเงินเข้าบัญชี B ถ้าแบตเตอรี่หมดในจังหวะที่หักเงินจากบัญชี A เสร็จแล้วแต่ยังไม่ได้เพิ่มให้บัญชี B เงินก้อนนั้นจะหายไปจากระบบ ถ้าสองคำสั่งนี้อยู่ใน transaction เดียวกัน เมื่อแอปเปิดขึ้นมาใหม่ SQLite จะพบว่า transaction นั้นยังไม่จบ และยกเลิก (rollback) ทุกสิ่งที่ทำไปแล้ว ยอดในบัญชี A จึงกลับมาเท่าเดิม

ข้อที่ต้องเข้าใจให้ถูกคือ SQLite รับประกันแบบนี้ให้โดยอัตโนมัติเฉพาะภายในคำสั่งเดียว ถ้าเราเรียก update() สองครั้งแยกกันโดยไม่ครอบด้วย transaction แต่ละคำสั่งจะถูกบันทึกทันทีที่ทำเสร็จ และกรณีแบตหมดระหว่างสองคำสั่งก็ยังทำให้เงินหายได้เหมือนเดิม การรับประกันข้ามหลายคำสั่งเกิดขึ้นก็ต่อเมื่อเราครอบคำสั่งเหล่านั้นด้วย transaction() เองเท่านั้น

เมธอด transaction() รับฟังก์ชันที่ทำงานกับอ็อบเจกต์ txn ถ้าฟังก์ชันจบตามปกติ ทุกคำสั่งจะถูกบันทึกพร้อมกัน ถ้ามีข้อผิดพลาดโยนออกมากลางทาง ทุกคำสั่งที่ทำไปแล้วในบล็อกนั้นจะถูกยกเลิก [15] ส่วน batch ช่วยรวมคำสั่งจำนวนมากส่งข้ามจาก Dart ไปยังฝั่งแพลตฟอร์มในรอบเดียว แทนที่จะส่งทีละคำสั่ง ซึ่งเร็วกว่ากันมากเมื่อมีหลายร้อยแถว

```dart
// ภายในคลาส DatabaseHelper
// แทนที่โน้ตทั้งหมดด้วยชุดใหม่ ถ้าล้มกลางทาง ข้อมูลชุดเดิมยังอยู่ครบ
Future<void> replaceAllNotes(List<Note> notes) async {
  final db = await database;
  await db.transaction((txn) async {
    await txn.delete('notes'); // ใช้ txn ทุกคำสั่งในบล็อกนี้ ห้ามใช้ db

    final batch = txn.batch();
    for (final note in notes) {
      batch.insert('notes', note.toMap());
    }
    // noResult: true เมื่อไม่ต้องการ id ของแต่ละแถว จะเร็วขึ้นอีก
    await batch.commit(noResult: true);
  });
}
```

กับดักที่ทำให้แอปค้าง ภายในบล็อกของ transaction ต้องใช้ตัวแปร txn ทุกคำสั่ง ถ้าเผลอเรียก db.insert() หรือเรียกเมธอดอื่นของ DatabaseHelper ที่ไปใช้ db ข้างใน คำสั่งนั้นจะรอให้ transaction จบก่อนจึงทำงาน แต่ transaction ก็รอคำสั่งนั้นอยู่ ทั้งสองฝ่ายจึงรอกันไปตลอดโดยไม่มีข้อผิดพลาดใด ๆ แสดงออกมา อาการที่เห็นคือหน้าจอค้างที่วงล้อหมุน

13.10 SQLite กับข้อมูลจากเครือข่าย: คำตอบของคำถามท้ายบทที่ 12

ตอนนี้เรามีเครื่องมือครบที่จะตอบคำถามสามข้อที่บทที่ 12 ฝากไว้

คำถามแรก ข้อมูลที่โหลดสำเร็จครั้งล่าสุดควรเก็บที่ไหน และใครเป็นผู้เก็บ คำตอบคือเก็บในตาราง SQLite บนตัวเครื่อง และผู้เก็บควรเป็นชั้นใหม่ที่อยู่ระหว่าง provider กับแหล่งข้อมูลทั้งสอง เรียกกันทั่วไปว่า repository [16] ชั้นนี้รู้จักทั้งฟังก์ชัน fetchTodos() จากบทที่ 12 และฐานข้อมูลในเครื่อง ส่วน provider รู้จักเพียง repository ไม่ต้องรู้ว่าข้อมูลมาจากเครือข่ายหรือจากไฟล์ ถ้าย้ายตรรกะนี้ไปไว้ใน fetchTodos() ชั้นเครือข่ายจะต้องรู้เรื่องฐานข้อมูลด้วย และถ้าไว้ใน provider โค้ดจะปนกันจนทดสอบแยกส่วนไม่ได้

คำถามที่สอง เมื่อมีข้อมูลสองชุด ชุดใดคือความจริง ในเชิงธุรกิจ เซิร์ฟเวอร์คือเจ้าของข้อมูล แต่สำหรับหน้าจอ แหล่งความจริงเพียงแหล่งเดียวควรเป็นฐานข้อมูลในเครื่อง หน้าจออ่านจากฐานข้อมูลเท่านั้น และข้อมูลจากเครือข่ายมีหน้าที่เดียวคือเขียนทับลงในฐานข้อมูล แนวทางนี้เรียกว่า offline-first ข้อดีคือหน้าจอมีข้อมูลให้แสดงทันทีตั้งแต่เสี้ยววินาทีแรก ไม่ว่าจะมีสัญญาณหรือไม่ และไม่มีจังหวะที่หน้าจอสองหน้าแสดงข้อมูลคนละชุด

มีข้อแย้งที่ควรตอบตรงนี้ ถ้าข้อมูลทั้งหมดอยู่ในเครื่อง เมื่อมือถือตกน้ำหรือเสียก็หายหมด ทำไมจึงยกการเก็บในเครื่องเป็นจุดเด่น คำตอบคือฐานข้อมูลในเครื่องไม่ได้มาแทนเซิร์ฟเวอร์ แต่มาทำหน้าที่เป็นกันชนระหว่างผู้ใช้กับเครือข่าย แอปที่พึ่งพาคลาวด์ร้อยเปอร์เซ็นต์จะกลายเป็นก้อนอิฐทันทีที่ผู้ใช้เดินเข้าลิฟต์หรือไปอยู่ในที่ไม่มีสัญญาณ ส่วนแอปที่มีฐานข้อมูลในเครื่องเปิดขึ้นมาใช้ได้ทันทีโดยไม่มีวงล้อหมุน และค่อยส่งหรือรับข้อมูลกับเซิร์ฟเวอร์เมื่อสัญญาณกลับมา สถาปัตยกรรมที่ให้ความสำคัญกับข้อมูลในเครื่องก่อนแบบนี้เรียกว่า local-first การสำรองข้อมูลขึ้นเซิร์ฟเวอร์ยังจำเป็นอยู่ แต่เป็นงานเบื้องหลัง ไม่ใช่สิ่งที่ผู้ใช้ต้องรอ

```dart
// lib/repositories/todo_repository.dart
import 'package:sqflite/sqflite.dart';
import '../models/todo.dart';
import '../services/api_service.dart';

class TodoRepository {
  TodoRepository(this._db);
  final Database _db;

  // อ่านชุดที่เก็บไว้ในเครื่อง ได้ทันทีแม้ไม่มีสัญญาณ
  Future<List<Todo>> readCached() async {
    final rows = await _db.query('todos', orderBy: 'id');
    return rows.map(Todo.fromMap).toList();
  }

  // ดึงชุดใหม่ เขียนทับในเครื่อง แล้วคืนสิ่งที่อยู่ในเครื่อง
  Future<List<Todo>> refresh() async {
    final fresh = await fetchTodos(); // ชั้นเครือข่ายจากหัวข้อ 12.6
    await _db.transaction((txn) async {
      await txn.delete('todos');
      final batch = txn.batch();
      for (final todo in fresh) {
        batch.insert('todos', todo.toMap());
      }
      await batch.commit(noResult: true);
    });
    return readCached(); // หน้าจอเห็นเฉพาะสิ่งที่อยู่ในฐานข้อมูล
  }
}
```

สังเกตว่า refresh() ใช้ transaction จากหัวข้อ 13.9 ถ้าแอปถูกปิดกลางทาง ข้อมูลชุดเดิมในเครื่องจะไม่หายไปครึ่งหนึ่ง และถ้า fetchTodos() ล้มเหลวตั้งแต่ต้น ฐานข้อมูลจะไม่ถูกแตะเลย ส่วนคลาส Todo ของบทที่ 12 ต้องเพิ่ม toMap() และ fromMap() ตามหลักในหัวข้อ 13.7 และสร้างตาราง todos ใน onCreate ให้มีคอลัมน์ตรงกัน

```
// lib/models/todo.dart (เพิ่มสองเมธอดนี้ในคลาส Todo ของบทที่ 12)
// ตารางที่คู่กัน: CREATE TABLE todos (id INTEGER PRIMARY KEY,
//   userId INTEGER NOT NULL, title TEXT NOT NULL, completed INTEGER NOT NULL)
Map<String, Object?> toMap() {
  return {
    'id': id, // id มาจากเซิร์ฟเวอร์ จึงมีค่าเสมอ
    'userId': userId,
    'title': title,
    'completed': completed ? 1 : 0, // SQLite ไม่มี bool
  };
}

factory Todo.fromMap(Map<String, Object?> map) {
  return Todo(
    userId: map['userId'] as int,
    id: map['id'] as int,
    title: map['title'] as String,
    completed: map['completed'] == 1,
  );
}
```

คำถามที่สาม สามสถานะยังพอหรือไม่ คำตอบคือไม่พอ เมื่อมีข้อมูลในเครื่อง สถานะกำลังโหลดแยกออกเป็นสองกรณีที่ต้องแสดงผลต่างกันมาก และสถานะผิดพลาดก็เช่นกัน จึงไม่ควรเก็บสถานะเป็นค่าเดียว แต่ควรเก็บเป็นหลายค่าที่ประกอบกันได้ คือรายการข้อมูล กำลังรีเฟรชอยู่หรือไม่ ข้อความผิดพลาดล่าสุด และเวลาที่ซิงก์สำเร็จครั้งล่าสุด

```
// lib/providers/todo_provider.dart
import 'package:flutter/foundation.dart';
import '../models/todo.dart';
import '../repositories/todo_repository.dart';
import '../services/api_service.dart';

class TodoProvider extends ChangeNotifier {
  TodoProvider(this._repository);
  final TodoRepository _repository;

  List<Todo> _todos = [];
  bool _isRefreshing = false;
  String? _errorMessage;
  DateTime? _lastSynced;

  List<Todo> get todos => List.unmodifiable(_todos);
  bool get isRefreshing => _isRefreshing;
```

```
  String? get errorMessage => _errorMessage;
  DateTime? get lastSynced => _lastSynced;

  Future<void> load() async {
    // 1. แสดงชุดในเครื่องก่อน
    _todos = await _repository.readCached();
    _isRefreshing = true;
    notifyListeners();

    // 2. ขอชุดใหม่ ถ้าไม่สำเร็จ ชุดเดิมยังอยู่บนจอ
    try {
      _todos = await _repository.refresh();
      _lastSynced = DateTime.now();
      _errorMessage = null;
    } catch (e) {
      _errorMessage = 'เกิดข้อผิดพลาด';
    } finally {
      _isRefreshing = false;
      notifyListeners();
    }
  }
}
```

ฝั่งหน้าจอเรียก load() ด้วย addPostFrameCallback แบบเดียวกับหัวข้อ 12.8 แล้วเลือกแสดงผลจากค่าทั้งสี่ประกอบกัน ซึ่งให้ผลสี่แบบ

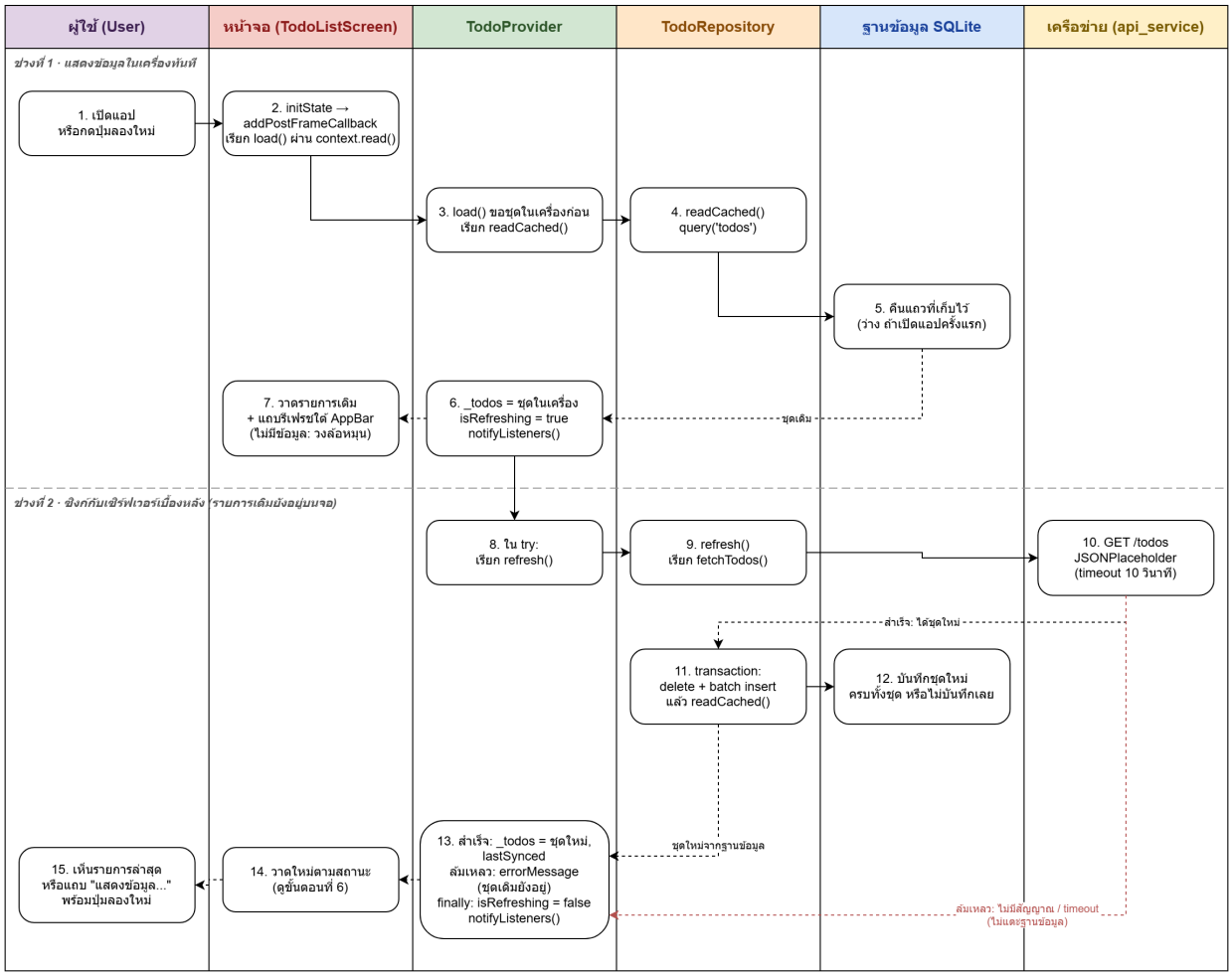
- ไม่มีข้อมูลและกำลังรีเฟรช คือเปิดแอปครั้งแรก แสดงวงล้อหมุนเต็มหน้าเหมือนบทที่ 12
- มีข้อมูลและกำลังรีเฟรช คือสถานะที่สี่ที่บทที่ 12 ถามถึง แสดงรายการเดิมตามปกติ พร้อมแถบความคืบหน้าบาง ๆ ใต้ AppBar ห้ามเปลี่ยนทั้งหน้าเป็นวงล้อหมุน
- มีข้อมูลแต่รีเฟรชไม่สำเร็จ แสดงรายการเดิมต่อไป พร้อมแถบข้อความเล็ก ๆ ว่าแสดงข้อมูลเมื่อเวลาใด ตามค่า lastSynced และปุ่มลองใหม่
- ไม่มีข้อมูลและรีเฟรชไม่สำเร็จ แสดงข้อความผิดพลาดพร้อมปุ่มลองใหม่เต็มหน้า

กลับไปที่ผู้ใช้ในลิฟต์ของบทที่ 12 ด้วยโครงสร้างนี้ เขาจะเห็นรายการที่โหลดไว้ตอนเช้าทันทีที่เปิดแอป พร้อมข้อความบอกว่าเป็นข้อมูลเมื่อเวลาเท่าใด และเมื่อออกจากลิฟต์แล้วกดลองใหม่ รายการจะอัปเดตโดยไม่กระพริบ ขอบเขตของหัวข้อนี้คือข้อมูลที่อ่านอย่างเดียวจากเซิร์ฟเวอร์ ถ้าผู้ใช้แก้ไขข้อมูลในเครื่องระหว่างออฟไลน์ด้วย จะเกิดคำถามใหม่ว่าต้องส่งการแก้ไขนั้นกลับไปอย่างไรและถ้าชนกับการแก้ไขของคนอื่นจะทำอย่างไร ซึ่งอยู่นอกขอบเขตของบทนี้

13.10.1 ประกอบทั้งหมดเป็นแอปที่รันได้

ก่อนลงมือประกอบไฟล์ ให้ดูภาพรวมก่อนว่าเมื่อทุกชิ้นต่อกันครบแล้ว ข้อมูลไหลผ่านส่วนใดบ้างตั้งแต่ผู้ใช้เปิดแอปจนเห็นรายการบนจอ แผนภาพต่อไปนี้แบ่งเป็น 6 lane ตามไฟล์ที่จะสร้าง เส้นทึบคือการเรียกใช้ เส้นประคือการส่งผลลัพธ์กลับหรือการแจ้งเตือน และเส้นประสีแดงคือเส้นทางเมื่อการเชื่อมต่อล้มเหลว

แผนภาพการทำงาน: แอป Todo แบบ offline-first (หัวข้อ 13.10.1)



ช่วงที่ 1 (ขั้นตอนที่ 1–7) แสดงข้อมูลในเครื่องทันที เมื่อหน้าจอถูกสร้าง TodoListScreen เรียก load() หลังเฟรมแรก provider ขอชุดที่เก็บไว้จาก repository ซึ่งอ่านจากตาราง todos ใน SQLite แล้วตั้ง isRefreshing เป็นจริงและเรียก notifyListeners() หน้าจอที่รับฟังอยู่ด้วย context.watch จึงวาดรายการเดิมพร้อมแถบรีเฟรชได้ในเสี้ยววินาทีโดยยังไม่ต้องรอเครือข่ายเลย ถ้าเปิดแอปเป็นครั้งแรก ชุดในเครื่องยังว่าง หน้าจอจึงแสดงวงล้อหมุนแทน

ช่วงที่ 2 (ขั้นตอนที่ 8–15) ซิงก์กับเซิร์ฟเวอร์เบื้องหลัง provider เรียก refresh() ซึ่ง repository ส่งต่อให้ fetchTodos() ยิงคำขอไปที่ JSONPlaceholder ถ้าสำเร็จ repository เขียนชุดใหม่ทับชุดเดิมภายใน transaction เดียว แล้วอ่านกลับจากฐานข้อมูลส่งให้ provider หน้าจอจึงเห็นเฉพาะสิ่งที่อยู่ในฐานข้อมูลเสมอ ตามหลักแหล่งความจริงเพียงแหล่งเดียวในหัวข้อ 13.10 ถ้าล้มเหลว เช่นไม่มีสัญญาณหรือเกินเวลาที่กำหนด ข้อผิดพลาดจะวิ่งตามเส้นสีแดงกลับมาที่ provider โดยไม่แตะฐานข้อมูลเลย ชุดเดิมจึงยังอยู่ครบ ไม่ว่าจะออกทางใด provider จะปิดสถานะรีเฟรชใน finally แล้วแจ้งหน้าจออีกครั้ง หน้าจอจึงเลือกแสดงผลตามสี่สถานะที่อธิบายไว้ก่อนหน้า

โค้ดในหัวข้อ 13.10 แสดงเฉพาะชั้น repository และ provider ซึ่งยังรันเป็นแอปไม่ได้ เพราะยังไม่มีส่วนที่เปิดฐานข้อมูลและสร้างตาราง todos ยังไม่มีหน้าจอ และยังไม่มี main.dart ที่ประกอบทุกชั้นเข้าด้วยกัน หัวข้อย่อยนี้เติมส่วนที่ขาดให้ครบ เมื่อทำตามทุกขั้นตอนแล้ว โปรเจกต์จะมีโครงสร้างไฟล์ดังนี้

```
lib/
  main.dart                          ขั้นตอนที่ 7
  data/app_database.dart             ขั้นตอนที่ 4
  models/todo.dart                   ขั้นตอนที่ 2
```

```
  services/api_service.dart           ขั้นตอนที่ 3
  repositories/todo_repository.dart   โค้ดด้านบน
  providers/todo_provider.dart        โค้ดด้านบน
  screens/todo_list_screen.dart       ขั้นตอนที่ 6
```

ขั้นตอนที่ 1 สร้างโปรเจกต์ใหม่ด้วยคำสั่ง flutter create todo_offline จากนั้นเพิ่ม dependencies ใน pubspec.yaml แล้วรัน flutter pub get

```
dependencies:
  flutter:
    sdk: flutter
  http: ^1.5.0          # จากบทที่ 12
  provider: ^6.1.5+1    # จากบทที่ 10
  sqflite: ^2.4.2
  path: ^1.9.1          # สำหรับ join()
```

ขั้นตอนที่ 2 สร้างไฟล์ Model ถ้าทำบทที่ 12 มาแล้ว ให้เพิ่ม toMap() และ fromMap() จากโค้ดด้านบนลงในคลาส Todo เดิม ถ้าเริ่มโปรเจกต์ใหม่ ให้ใช้ไฟล์ฉบับเต็มต่อไปนี้

```dart
// lib/models/todo.dart (ฉบับเต็ม: Todo ของบทที่ 12 รวมกับ toMap/fromMap ด้านบน)
class Todo {
  final int userId;
  final int id;
  final String title;
  final bool completed;

  const Todo({
    required this.userId,
    required this.id,
    required this.title,
    required this.completed,
  });

  // แปลงจาก JSON ที่เซิร์ฟเวอร์ส่งมา (บทที่ 12)
  factory Todo.fromJson(Map<String, dynamic> json) {
    return Todo(
      userId: json['userId'] as int,
      id: json['id'] as int,
      title: json['title'] as String,
      completed: json['completed'] as bool,
    );
  }

  // แปลงเป็นแถวของตาราง todos (หัวข้อ 13.7)
  Map<String, Object?> toMap() {
    return {
      'id': id,
```

```
      'userId': userId,
      'title': title,
      'completed': completed ? 1 : 0,
    };
  }

  factory Todo.fromMap(Map<String, Object?> map) {
    return Todo(
      userId: map['userId'] as int,
      id: map['id'] as int,
      title: map['title'] as String,
      completed: map['completed'] == 1,
    );
  }
}
```

ขั้นตอนที่ 3 คัดลอกไฟล์ api_service.dart ฉบับรับมือความล้มเหลวจากหัวข้อ 12.6 มาทั้งไฟล์ ไฟล์นี้มีฟังก์ชัน fetchTodos() ที่ repository เรียกใช้

```
// lib/services/api_service.dart (คัดลอกฉบับรับมือความล้มเหลวจากหัวข้อ 12.6 มาทั้งไฟล์)
import 'dart:async'; // สำหรับ TimeoutException
import 'dart:convert';
import 'package:http/http.dart' as http;
import '../models/todo.dart';

// ข้อผิดพลาดที่แอปเข้าใจเอง พร้อมข้อความที่นำไปแสดงต่อผู้ใช้ได้ทันที
class ApiException implements Exception {
  final String message; // ข้อความภาษาคน ไม่ใช่ข้อความของระบบ
  final bool canRetry;  // ปุ่มลองใหม่ควรมีประโยชน์หรือไม่
  const ApiException(this.message, {this.canRetry = true});

  @override
  String toString() => message;
}

Future<List<Todo>> fetchTodos() async {
  final uri = Uri.parse('https://jsonplaceholder.typicode.com/todos');
  try {
    // http ไม่มี timeout ให้โดยปริยาย ต้องกำหนดเองทุกคำขอ
    final response = await http
        .get(uri, headers: {'Accept': 'application/json'})
        .timeout(const Duration(seconds: 10));

    if (response.statusCode == 200) {
      final data = jsonDecode(response.body) as List<dynamic>;
      return data
          .map((item) => Todo.fromJson(item as Map<String, dynamic>))
```

```
          .toList();
    }
    if (response.statusCode >= 500) {
      // ผิดที่ฝั่งเซิร์ฟเวอร์ การลองใหม่มีโอกาสสำเร็จ
      throw const ApiException('เซิร์ฟเวอร์ขัดข้องชั่วคราว โปรดลองอีกครั้ง');
    }
    // 4xx คือคำขอของเราเองไม่ถูกต้อง ลองใหม่ด้วยคำขอเดิมก็ได้ผลเดิม
    throw ApiException(
      'ไม่พบข้อมูลที่ร้องขอ (รหัส ${response.statusCode})',
      canRetry: false,
    );
  } on TimeoutException {
    throw const ApiException('เซิร์ฟเวอร์ตอบกลับช้าเกินไป โปรดลองอีกครั้ง');
  } on FormatException {
    // คำตอบไม่ใช่ JSON เช่นโดนหน้าล็อกอินของเครือข่ายขั้นกลางคั่นไว้
    throw const ApiException('รูปแบบข้อมูลที่ได้รับไม่ถูกต้อง', canRetry: false);
  } on ApiException {
    rethrow; // โยนต่อ ไม่ให้ตัวดักบรรทัดล่างกลืนข้อความที่เราตั้งใจไว้
  } catch (_) {
    // เหลือกรณีต่อไม่ติด เช่นไม่มีสัญญาณ หรือหาชื่อโฮสต์ไม่พบ
    throw const ApiException('เชื่อมต่ออินเทอร์เน็ตไม่ได้ โปรดตรวจสอบการเชื่อมต่อ');
  }
}
```

ขั้นตอนที่ 4 สร้างไฟล์สำหรับเปิดฐานข้อมูล ตัวอย่างนี้ใช้ฟังก์ชันเดียวแทนคลาส DatabaseHelper ในหัวข้อ 13.4 เพื่อให้สั้นลง แต่หลักการเหมือนกัน คือเปิดไฟล์ครั้งเดียวแล้วส่งอ็อบเจกต์ Database ตัวเดียวกันให้ทุกส่วนของแอปใช้ สังเกตว่าคอลัมน์ id ไม่มี AUTOINCREMENT เพราะค่า id มาจากเซิร์ฟเวอร์

```
// lib/data/app_database.dart
import 'package:path/path.dart';
import 'package:sqflite/sqflite.dart';

// เปิดไฟล์ฐานข้อมูลของแอป และสร้างตาราง todos ในครั้งแรกที่ไฟล์ถูกสร้าง
Future<Database> openAppDatabase() async {
  final path = join(await getDatabasesPath(), 'todos.db');
  return openDatabase(
    path,
    version: 1,
    onCreate: (db, version) {
      // id มาจากเซิร์ฟเวอร์ จึงไม่ใช้ AUTOINCREMENT
      return db.execute('''
        CREATE TABLE todos (
          id INTEGER PRIMARY KEY,
          userId INTEGER NOT NULL,
          title TEXT NOT NULL,
          completed INTEGER NOT NULL
        )
```

```
      ''');
    },
  );
}
```

ขั้นตอนที่ 5 สร้างไฟล์ todo_repository.dart และ todo_provider.dart ด้วยโค้ดที่ให้ไว้ก่อนหน้าในหัวข้อ 13.10 โดยไม่ต้องแก้ไข

ขั้นตอนที่ 6 สร้างหน้าจอที่เลือกแสดงผลตามสี่สถานะข้างต้น เงื่อนไขสองข้อแรกใน _buildBody() รับกรณีที่ยังไม่มีข้อมูลเลย ส่วนกรณีที่มีข้อมูลจะแสดงรายการเสมอ โดยมีแถบความคืบหน้าบาง ๆ ใต้ AppBar ระหว่างรีเฟรช และมี MaterialBanner บอกที่มาของข้อมูลพร้อมปุ่มลองใหม่เมื่อรีเฟรชไม่สำเร็จ

```
// lib/screens/todo_list_screen.dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import '../providers/todo_provider.dart';

class TodoListScreen extends StatefulWidget {
  const TodoListScreen({super.key});

  @override
  State<TodoListScreen> createState() => _TodoListScreenState();
}

class _TodoListScreenState extends State<TodoListScreen> {
  @override
  void initState() {
    super.initState();
    // load() เรียก notifyListeners() ทันที จึงเลื่อนไปหลังเฟรมแรก (หัวข้อ 12.8)
    WidgetsBinding.instance.addPostFrameCallback((_) {
      if (!mounted) return;
      context.read<TodoProvider>().load();
    });
  }

  @override
  Widget build(BuildContext context) {
    final provider = context.watch<TodoProvider>();
    final hasData = provider.todos.isNotEmpty;

    return Scaffold(
      appBar: AppBar(
        title: const Text('Todos (offline-first)'),
        // สถานะที่สี่: มีข้อมูลและกำลังรีเฟรช แสดงแถบบาง ๆ ใต้ AppBar
        bottom: hasData && provider.isRefreshing
            ? const PreferredSize(
                preferredSize: Size.fromHeight(4),
                child: LinearProgressIndicator(),
```

```
            )
          : null,
    ),
    body: _buildBody(provider, hasData),
  );
}

Widget _buildBody(TodoProvider provider, bool hasData) {
  // ไม่มีข้อมูลและกำลังรีเฟรช: เปิดแอปครั้งแรก
  if (!hasData && provider.isRefreshing) {
    return const Center(child: CircularProgressIndicator());
  }
  // ไม่มีข้อมูลและรีเฟรชไม่สำเร็จ: ข้อความผิดพลาดพร้อมปุ่มลองใหม่เต็มหน้า
  if (!hasData && provider.errorMessage != null) {
    return Center(
      child: Column(
        mainAxisSize: MainAxisSize.min,
        children: [
          Text(provider.errorMessage!, textAlign: TextAlign.center),
          const SizedBox(height: 12),
          ElevatedButton(
            onPressed: provider.load,
            child: const Text('ลองใหม่'),
          ),
        ],
      ),
    );
  }
  // มีข้อมูล: แสดงรายการเดิมเสมอ ถ้ารีเฟรชไม่สำเร็จให้มีแถบบอกที่มาของข้อมูล
  return Column(
    children: [
      if (provider.errorMessage != null)
        MaterialBanner(
          content: Text(
            'แสดงข้อมูล${_syncedText(provider.lastSynced)}\n'
            '${provider.errorMessage}',
          ),
          actions: [
            TextButton(
              onPressed: provider.load,
              child: const Text('ลองใหม่'),
            ),
          ],
        ),
      Expanded(
        child: ListView.builder(
          itemCount: provider.todos.length,
```

```
            itemBuilder: (context, index) {
              final todo = provider.todos[index];
              return CheckboxListTile(
                value: todo.completed,
                onChanged: null, // แสดงอย่างเดียว ไม่ให้แก้ไข
                title: Text(todo.title),
              );
            },
          ),
        ),
      ],
    );
  }

  // lastSynced เป็น null เมื่อเปิดแอปใหม่แล้วยังซิงก์ไม่สำเร็จเลย
  String _syncedText(DateTime? synced) {
    if (synced == null) return 'ที่บันทึกไว้ในเครื่องจากครั้งก่อน';
    return 'เมื่อ ${TimeOfDay.fromDateTime(synced).format(context)}';
  }
}
```

ขั้นตอนที่ 7 แก้ไข main.dart ให้เปิดฐานข้อมูลก่อนเริ่มแอป แล้วส่ง Database ต่อให้ repository และ provider ตามลำดับ คำสั่ง WidgetsFlutterBinding.ensureInitialized() ต้องอยู่ก่อนการเรียก plugin ใด ๆ ที่ทำก่อน runApp() ไม่เช่นนั้นแอปจะหยุดทำงานตั้งแต่เปิด

```
// lib/main.dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'data/app_database.dart';
import 'providers/todo_provider.dart';
import 'repositories/todo_repository.dart';
import 'screens/todo_list_screen.dart';

Future<void> main() async {
  // ต้องเรียกก่อนใช้ plugin ใด ๆ ก่อน runApp เพราะ sqflite ต้องคุยกับฝั่งแพลตฟอร์ม
  WidgetsFlutterBinding.ensureInitialized();
  final db = await openAppDatabase(); // เปิดครั้งเดียวตลอดอายุของแอป

  runApp(
    ChangeNotifierProvider(
      create: (_) => TodoProvider(TodoRepository(db)),
      child: const MyApp(),
    ),
  );
}

class MyApp extends StatelessWidget {
```

```
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      title: 'Offline-first Todos',
      home: TodoListScreen(),
    );
  }
}
```

ขั้นตอนที่ 8 รันบน emulator หรือเครื่องจริง แล้วทดสอบให้เห็นครบทั้งสี่สถานะตามลำดับต่อไปนี้

1) เปิดอินเทอร์เน็ตแล้วรันแอป จะเห็นวงล้อหมุนชั่วครู่ แล้วตามด้วยรายการ 200 รายการจาก JSONPlaceholder

2) เปิดโหมดเครื่องบิน ปิดแอปแล้วรันใหม่ รายการเดิมแสดงขึ้นมาทันทีโดยไม่มีวงล้อหมุนเต็มหน้า พร้อมแถบข้อความว่าเป็นข้อมูลที่บันทึกไว้ในเครื่องจากครั้งก่อน

3) ปิดโหมดเครื่องบินแล้วกดลองใหม่ จะเห็นแถบความคืบหน้าบาง ๆ ใต้ AppBar ขณะที่รายการเดิมยังอยู่บนจอ และแถบข้อความหายไปเมื่อรีเฟรชสำเร็จ

4) ถอนการติดตั้งแอปเพื่อล้างฐานข้อมูล เปิดโหมดเครื่องบิน แล้วรันใหม่ จะเห็นข้อความผิดพลาดพร้อมปุ่มลองใหม่เต็มหน้า

หากนำไปสร้างฉบับ release อย่าลืมประกาศสิทธิ์ INTERNET ในไฟล์ manifest ของ main ตามหัวข้อ 12.5 ไม่เช่นนั้นแอปจะติดอยู่ที่สถานะที่สี่ตลอดแม้เครื่องจะต่ออินเทอร์เน็ตอยู่

13.11 สรุปบทเรียนและสิ่งที่จะเรียนรู้ต่อไป

บทนี้ได้แนะนำวิธีการจัดเก็บข้อมูลที่มีโครงสร้างอย่างถาวรบนอุปกรณ์โดยใช้ SQLite ใน Flutter:

- SQLite: ฐานข้อมูลเชิงสัมพันธ์แบบ file-based ที่เหมาะสำหรับมือถือ.
- sqflite & path_provider: Dependencies หลักสำหรับการทำงานกับ SQLite.
- Database Helper & Singleton: รูปแบบทั่วไปสำหรับการจัดการตรรกะฐานข้อมูล.
- การเปิดฐานข้อมูล: ใช้ openDatabase, path_provider, และ callback onCreate.
- CREATE TABLE: การกำหนด schema ฐานข้อมูลโดยใช้ SQL.
- Model Classes: การสร้างคลาส Dart (toMap, fromMap) เพื่อแทนข้อมูลในตาราง.
- CRUD Operations: การใช้ db.insert(), db.query(), db.update(), db.delete().
- Asynchronous Nature: การดำเนินการฐานข้อมูลทั้งหมดเป็น asynchronous และต้องใช้ async/await.
- SQL Injection: ค่าที่มาจากผู้ใช้หรือเครือข่ายต้องส่งผ่าน ? กับ whereArgs เสมอ ห้ามต่อเข้าไปในสตริงของคำสั่ง SQL.
- การแปลงชนิดข้อมูล: SQLite ไม่มี bool และ DateTime ให้ตกลงวิธีแปลงไว้ใน toMap และ fromMap จุดเดียว และใส่ id ใน toMap เฉพาะเมื่อมีค่า.
- Migration: เพิ่ม version แล้วเขียน onUpgrade แบบ if (oldVersion < N) เรียงต่อกัน การถอนการติดตั้งแอปใช้ได้บนเครื่องนักพัฒนาเท่านั้น.
- Transaction และ batch: คำสั่งที่ต้องสำเร็จไปด้วยกันให้อยู่ใน transaction และใช้ txn ทุกคำสั่งในบล็อก ไม่เช่นนั้นแอปจะค้าง.
- Offline-first: หน้าจออ่านจากฐานข้อมูลในเครื่องเท่านั้น ข้อมูลจากเครือข่ายเขียนทับลงฐานข้อมูลผ่าน repository และสถานะของหน้าจอเก็บเป็นหลายค่าประกอบกัน ไม่ใช่ค่าเดียว.

ตอนนี้คุณมีความสามารถในการเพิ่มชั้นการคงอยู่ของข้อมูล (persistence layer) ให้กับแอป Flutter ของคุณแล้ว.

บทต่อไป:

เราได้สร้างส่วนประกอบสำคัญเกือบทั้งหมดแล้ว: UI, การจัดการ state, networking, และ data persistence. บทต่อไป บทที่ 14: โปรเจกต์ส่วนที่ 1 - การวางแผนและตั้งค่า UI จะเป็นการเริ่มต้นโปรเจกต์ขนาดเล็ก (เช่น Task Manager หรือ Note Taking App) โดยเราจะวางแผน features, ออกแบบ UI พื้นฐาน, และตั้งค่าโครงสร้างโปรเจกต์.

13.12 คำถามชวนคิดท้ายบท

ลองเทียบสถาปัตยกรรมสองแบบที่ปรากฏในบทนี้ เนื้อหาหลักแยกงานฐานข้อมูลไว้ใน DatabaseHelper แบบ Singleton ขณะที่ปฏิบัติการท้ายบทให้ TransactionProvider เปิดและถือฐานข้อมูลเอง ทั้งสองแบบทำงานได้ถูกต้องสำหรับแอปที่มีหน้าจอเดียว

คำถามคือ เมื่อโปรเจกต์ในบทที่ 14 และ 15 เติบโตขึ้นจนมี provider สองตัวที่ต้องใช้ฐานข้อมูลไฟล์เดียวกัน เช่นตัวหนึ่งดูแลรายการงานและอีกตัวดูแลหมวดหมู่ของงาน แบบใดจะทำให้ไฟล์ฐานข้อมูลถูกเปิดเพียงครั้งเดียวและมีผู้ดูแล onUpgrade เพียงคนเดียว และถ้าต้องการทดสอบตรรกะของ provider โดยไม่แตะไฟล์ฐานข้อมูลจริงเลย ควรแบ่งชั้นของโค้ดอย่างไร

และคำถามที่ยากที่สุดซึ่งหัวข้อ 13.10 ยังไม่ได้ตอบ ถ้าแอปต้องสำรองข้อมูลขึ้นเซิร์ฟเวอร์ด้วย สมมติว่าผู้ใช้มีมือถือสองเครื่อง เปิดแอปตอนไม่มีสัญญาณทั้งคู่แล้วแก้ไขรายการเดียวกันคนละแบบ เมื่อทั้งสองเครื่องกลับมาออนไลน์ ข้อมูลจากเครื่องใดควรเป็นฉบับจริง เครื่องที่ต่อเน็ตได้ก่อน หรือเครื่องที่แก้ไขล่าสุด และถ้าเลือกเครื่องที่แก้ไขล่าสุด นาฬิกาของสองเครื่องเชื่อถือได้แค่ไหน ปัญหานี้เรียกว่า conflict resolution ซึ่งเป็นหนึ่งในโจทย์ที่ท้าทายที่สุดของแอปที่ทำงานแบบออฟไลน์

ลองตอบด้วยตัวเองก่อน จากนั้นนำคำตอบไปเทียบกับโครงสร้างที่บทที่ 15 ใช้ ซึ่งให้ TaskProvider เรียกใช้ DatabaseHelper อีกทอดหนึ่ง

เอกสารอ้างอิง (References)

[1] Flutter Team, "Store key-value data on disk," *Flutter Documentation*. Accessed: Sep. 24, 2026. [Online]. Available: https://docs.flutter.dev/cookbook/persistence/key-value

[2] Flutter Team, "Persist data with SQLite," *Flutter Documentation*. Accessed: Sep. 24, 2026. [Online]. Available: https://docs.flutter.dev/cookbook/persistence/sqlite

[3] Tekartik, "sqflite package," *pub.dev*. Accessed: Apr. 30, 2025. [Online]. Available: https://pub.dev/packages/sqflite

[4] Flutter Team, "path_provider package," *pub.dev*. Accessed: Apr. 30, 2025. [Online]. Available: https://pub.dev/packages/path_provider

[5] SQLite Consortium, "About SQLite," *SQLite Documentation*. Accessed: Apr. 30, 2025. [Online]. Available: https://www.sqlite.org/about.html

[6] SQLite Consortium, "Datatypes In SQLite," *SQLite Documentation*. Accessed: Sep. 24, 2026. [Online]. Available: https://www.sqlite.org/datatype3.html

[7] Tekartik, "sqflite: Supported types," *GitHub*. Accessed: Sep. 24, 2026. [Online]. Available: https://github.com/tekartik/sqflite/blob/master/sqflite/doc/supported_types.md

[8] SQLite Consortium, "SQL As Understood By SQLite - CREATE TABLE," *SQLite Documentation*. Accessed: Apr. 30, 2025. [Online]. Available: https://www.sqlite.org/lang_createtable.html

[9] Tekartik, "sqflite package - Insert helper," *pub.dev*. Accessed: Apr. 30, 2025. [Online]. Available: https://pub.dev/documentation/sqflite/latest/sqflite/DatabaseExecutor/insert.html

[10] Tekartik, "sqflite package - Query helper," *pub.dev*. Accessed: Apr. 30, 2025. [Online]. Available: https://pub.dev/documentation/sqflite/latest/sqflite/DatabaseExecutor/query.html

[11] Tekartik, "sqflite package - Update helper," *pub.dev*. Accessed: Apr. 30, 2025. [Online]. Available: https://pub.dev/documentation/sqflite/latest/sqflite/DatabaseExecutor/update.html

[12] Tekartik, "sqflite package - Delete helper," *pub.dev*. Accessed: Apr. 30, 2025. [Online]. Available: https://pub.dev/documentation/sqflite/latest/sqflite/DatabaseExecutor/delete.html

[13] Tekartik, "sqflite: Migration example," *GitHub*. Accessed: Sep. 24, 2026. [Online]. Available: https://github.com/tekartik/sqflite/blob/master/sqflite/doc/migration_example.md

[14] SQLite Consortium, "ALTER TABLE," *SQLite Documentation*. Accessed: Sep. 24, 2026. [Online]. Available: https://www.sqlite.org/lang_altertable.html

[15] Tekartik, "sqflite: SQL helpers (transaction)," *GitHub*. Accessed: Sep. 24, 2026. [Online]. Available: https://github.com/tekartik/sqflite/blob/master/sqflite/doc/sql.md

[16] Flutter Team, "Offline-first support," *Flutter Documentation*. Accessed: Sep. 24, 2026. [Online]. Available: https://docs.flutter.dev/app-architecture/design-patterns/offline-first

ปฏิบัติการบทที่ 13: แอปบัญชีรายรับ-รายจ่ายด้วย Provider และ SQLite

ในปฏิบัติการนี้ เราจะนำความรู้เรื่อง SQLite มารวมกับ Provider เพื่อสร้างแอปพลิเคชันจัดการรายรับ-รายจ่ายส่วนบุคคลที่สมบูรณ์ยิ่งขึ้น โดยจะแสดงให้เห็นถึงสถาปัตยกรรมที่ Provider ทำหน้าที่เป็นศูนย์กลางในการจัดการทั้งสถานะ (State) และการสื่อสารกับฐานข้อมูลโดยตรง แทนการใช้คลาส Helper แบบ Singleton

ก่อนเริ่ม ลองนึกภาพสถาปัตยกรรมของแอปนี้เป็นบริษัทที่แบ่งแผนกชัดเจน หน้าจอหรือวิดเจ็ตคือพนักงานต้อนรับ มีหน้าที่รับเรื่องจากผู้ใช้และแสดงผล แต่ไม่ลงมือจัดการข้อมูลเอง TransactionProvider คือผู้จัดการที่รู้ตรรกะทางธุรกิจทั้งหมด เมื่อได้รับเรื่องจะสั่งลงไปที่ห้องใต้ดินซึ่งก็คือ SQLite ให้เก็บหรือเบิกเอกสาร เมื่อห้องใต้ดินแจ้งว่าเสร็จ ผู้จัดการจะดึงข้อมูลล่าสุดมาถือไว้ แล้วประกาศผ่านลำโพงที่ชื่อ notifyListeners() พนักงานต้อนรับที่ใส่หูฟังรอผ่าน Consumer ได้ยินแล้วก็วาดหน้าจอใหม่ตามข้อมูลที่ผู้จัดการถืออยู่

หลักการนี้เรียกว่า Separation of Concerns หรือการแยกความรับผิดชอบ หน้าจอไม่ต้องรู้เลยว่ามีภาษา SQL อยู่ในโลก และ provider ก็ไม่ต้องสนใจว่าปุ่มบนจอเป็นสีอะไร เหตุผลของการแยกไม่ใช่เพราะหน้าจอจะค้างถ้าเรียกฐานข้อมูลเอง เพราะคำสั่งของ sqflite เป็นอะซิงโครนัสและไม่ขวางการวาดหน้าจออยู่แล้ว เหตุผลที่แท้จริงคือเมื่อหลายหน้าจอใช้ข้อมูลชุดเดียวกัน ต้องมีผู้ถือข้อมูลเพียงคนเดียวที่แจ้งทุกหน้าจอพร้อมกันได้ และเมื่อวันหนึ่งต้องเปลี่ยนวิธีเก็บข้อมูล เราแก้เฉพาะหลังบ้านโดยไม่ต้องแตะโค้ดหน้าจอแม้แต่บรรทัดเดียว

หมายเหตุเรื่องสถาปัตยกรรม: การให้ provider ถือฐานข้อมูลเองทำให้ปฏิบัติการนี้สั้นและเห็นภาพการไหลของข้อมูลครบในไฟล์เดียว แต่เมื่อแอปมี provider หลายตัวที่ใช้ฐานข้อมูลเดียวกัน ควรกลับไปแยกงานฐานข้อมูลไว้ในชั้นของตัวเองแบบ DatabaseHelper ในเนื้อหาหลัก ดูคำถามชวนคิดในหัวข้อ 13.12

กระบวนการที่ 1: การเตรียมโปรเจกต์และภาพรวม

เป้าหมายของกระบวนการ

เพื่อเตรียมสภาพแวดล้อมสำหรับการพัฒนาแอปพลิเคชันทั้งหมด โดยการสร้างโปรเจกต์ใหม่, ติดตั้งเครื่องมือ (dependencies) ที่จำเป็น, และทำความเข้าใจภาพรวมของสถาปัตยกรรมที่เราจะสร้างขึ้น ซึ่งจะช่วยให้เห็นว่าแต่ละส่วนของโค้ดจะเชื่อมต่อและทำงานร่วมกันอย่างไรตั้งแต่ต้นจนจบ

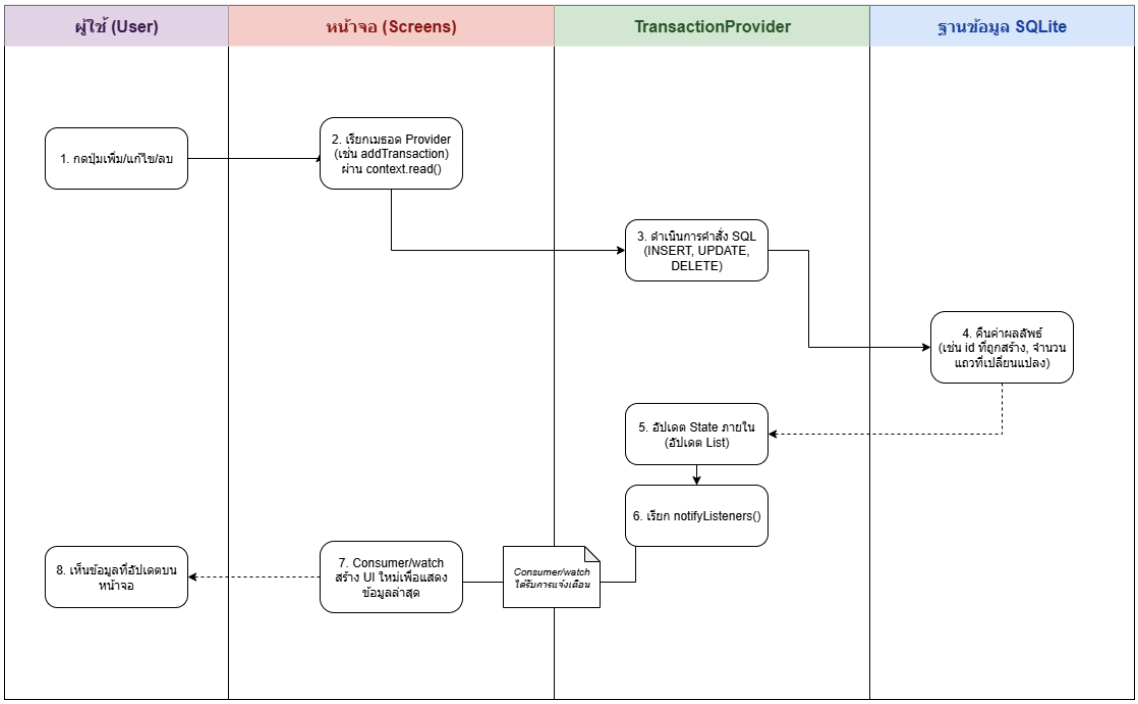
- ความสามารถในแอป: ยังไม่มีความสามารถที่ผู้ใช้มองเห็นได้ แต่เป็นขั้นตอนการวางโครงสร้างพื้นฐานที่จำเป็น
- การเชื่อมต่อ: pubspec.yaml จะเป็นไฟล์หลักที่เราทำงานด้วย
- Input/Output: Input คือชื่อแพ็กเกจที่เราต้องการ, Output คือการที่แพ็กเกจเหล่านั้นพร้อมใช้งานในโปรเจกต์

1. คำแนะนำ: สร้างโปรเจกต์ Flutter ใหม่ชื่อ expense_tracker_provider
2. คำแนะนำ: เปิด pubspec.yaml และเพิ่ม dependencies ที่จำเป็น: sqflite, path, provider และ intl (สำหรับจัดรูปแบบวันที่)

```
dependencies:
  flutter:
    sdk: flutter
  sqflite: ^2.4.2
  path: ^1.9.1          # สำหรับ join() ใน transaction_provider.dart
  provider: ^6.1.5+1
  intl: ^0.20.2         # สำหรับ DateFormat
```

3. คำแนะนำ: รัน flutter pub get
4. ภาพรวมสถาปัตยกรรม (Swimlane Diagram): แผนภาพนี้แสดงให้เห็นว่าส่วนต่างๆ ของแอปจะทำงานร่วมกันอย่างไร

แผนภาพกระบวนการ: แอปบัญชีรายรับ-รายจ่าย



กระบวนการที่ 2: การสร้าง Model และ Provider จัดการฐานข้อมูล

เป้าหมายของกระบวนการ

เพื่อกำหนดโครงสร้างข้อมูลของแอป (Model) และสร้างศูนย์กลางการจัดการตรรกะ (Provider) ที่จะรับผิดชอบการเชื่อมต่อและสร้างฐานข้อมูล SQLite โค้ดในส่วนนี้จะเป็นแกนหลักที่จัดการกับข้อมูลทั้งหมด

- ความสามารถในแอป: สร้างไฟล์ฐานข้อมูล .db ขึ้นมาในเครื่องของผู้ใช้โดยอัตโนมัติเมื่อแอปถูกเปิดใช้งานครั้งแรก
- การเชื่อมต่อ: TransactionProvider จะใช้ getDatabasesPath() ของ sqflite เพื่อหาโฟลเดอร์ฐานข้อมูลมาตรฐานของเครื่อง และใช้ join() ของแพ็กเกจ path ต่อชื่อไฟล์ ปฏิบัติการนี้จึงไม่ต้องใช้ path_provider
- Input/Output: Input คือโครงสร้างตารางที่เรากำหนดในโค้ด (CREATE TABLE), Output คือไฟล์ expenses.db ที่พร้อมใช้งาน

1. คำแนะนำ: สร้างไฟล์ lib/models/my_transaction.dart

```dart
// lib/models/my_transaction.dart
enum TransactionType { income, expense }

class MyTransaction {
  final int? id;
  final String title;
  final double amount;
  final DateTime date;
```

```
  final TransactionType type;

  MyTransaction({
    this.id,
    required this.title,
    required this.amount,
    required this.date,
    required this.type,
  });

  Map<String, dynamic> toMap() {
    return {
      if (id != null) 'id': id, // ห้ามส่ง id ที่เป็น null เข้าไปใน UPDATE
      'title': title,
      'amount': amount,
      'date': date.toIso8601String(),
      'type': type.name, // เก็บเป็น 'income' หรือ 'expense'
    };
  }

  factory MyTransaction.fromMap(Map<String, dynamic> map) {
    return MyTransaction(
      id: map['id'] as int,
      title: map['title'] as String,
      amount: (map['amount'] as num).toDouble(),
      date: DateTime.parse(map['date'] as String),
      type: TransactionType.values.byName(map['type'] as String),
    );
  }
}
```

สามจุดใน Model นี้ตามหลักของหัวข้อ 13.7 คือ id ถูกใส่ใน toMap() เฉพาะเมื่อมีค่า ถ้าไม่ทำแบบนี้ updateTransaction ในกระบวนการที่ 5 จะล้มด้วยข้อผิดพลาด datatype mismatch ต่อมา type เก็บด้วย .name ได้คำว่า income หรือ expense ซึ่งอ่านง่ายและไม่ผูกกับชื่อ enum และ amount แปลงผ่าน num เพื่อรองรับแถวที่อาจถูกบันทึกเป็นจำนวนเต็ม

2. คำแนะนำ: สร้างไฟล์ lib/providers/transaction_provider.dart นี่จะเป็นหัวใจของแอปเรา

```
// lib/providers/transaction_provider.dart
import 'package:flutter/foundation.dart';
import 'package:path/path.dart';
import 'package:sqflite/sqflite.dart';
import '../models/my_transaction.dart';
```

```dart
class TransactionProvider with ChangeNotifier {
  static const String _dbName = 'expenses.db';
  static const String _tableName = 'transactions';
  Database? _database;
  List<MyTransaction> _transactions = [];

  List<MyTransaction> get transactions => [..._transactions];

  // กระบวนการที่ 2: การสร้างฐานข้อมูล
  Future<void> _initDatabase() async {
    if (_database != null) return;
    try {
      final dbPath = await getDatabasesPath();
      final path = join(dbPath, _dbName);
      _database = await openDatabase(
        path,
        version: 1,
        onCreate: (db, version) {
          print('Creating table $_tableName...');
          return db.execute(
            'CREATE TABLE $_tableName(id INTEGER PRIMARY KEY AUTOINCREMENT, title
TEXT, amount REAL, date TEXT, type TEXT)',
          );
        },
      );
      print('Database initialized at $path');
    } catch (e) {
      print('Error initializing database: $e');
    }
  }
}
```

การทดสอบ: เราจะทดสอบการสร้างฐานข้อมูลในขั้นตอนถัดไปเมื่อเราเชื่อมต่อ Provider เข้ากับแอป

กระบวนการที่ 3: การเพิ่มข้อมูล (Insert)

เป้าหมายของกระบวนการ

เพื่อเพิ่มความสามารถให้แอปสามารถ บันทึกข้อมูลใหม่ ที่ผู้ใช้ป้อนลงในฐานข้อมูลได้ กระบวนการนี้เชื่อมโยงการกระทำของผู้ใช้บน UI กับการทำงานของฐานข้อมูลเป็นครั้งแรก

- ความสามารถในแอป: เมื่อผู้ใช้กรอกข้อมูลในฟอร์มและกดบันทึก ข้อมูลนั้นจะถูกเพิ่มลงในตาราง transactions ในไฟล์ SQLite
- การเชื่อมต่อ: หน้าจอ AddEditTransactionScreen (ที่เราจะสร้างในภายหลัง) จะเรียกเมธอด addTransaction ใน TransactionProvider, ซึ่ง TransactionProvider จะเรียกใช้คำสั่ง INSERT ของ sqflite

- Input/Output: Input คือข้อมูลรายการ (title, amount, date, type) ที่รับมาจาก UI, Output คือ id ของแถวใหม่ที่ถูกสร้างขึ้นในฐานข้อมูล

1. คำแนะนำ: เพิ่มเมธอด addTransaction ใน TransactionProvider

```
// ภายในคลาส TransactionProvider
// กระบวนการที่ 3: การ Insert
Future<void> addTransaction(String title, double amount, DateTime date,
TransactionType type) async {
  await _initDatabase(); // ตรวจสอบว่า DB พร้อมใช้งาน
  if (_database == null) return;

  final newTransaction = MyTransaction(
    title: title,
    amount: amount,
    date: date,
    type: type,
  );

  final id = await _database!.insert(_tableName, newTransaction.toMap());
  print('Inserted transaction with id: $id');
  // await fetchAndSetTransactions(); // เปิดบรรทัดนี้ในกระบวนการที่ 4
}
```

2. การทดสอบ: สร้าง UI ชั่วคราวใน lib/main.dart เพื่อทดสอบการ Insert

```
// lib/main.dart (ชั่วคราวสำหรับทดสอบ)
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'providers/transaction_provider.dart';
import 'models/my_transaction.dart';

void main() {
  runApp(
    ChangeNotifierProvider(
      create: (context) => TransactionProvider(),
      child: const MyApp(),
    ),
  );
}
class MyApp extends StatelessWidget {
  const MyApp({super.key});
  @override
  Widget build(BuildContext context) {
    return const MaterialApp(home: TestScreen());
  }
}
class TestScreen extends StatelessWidget {
  const TestScreen({super.key});
```

```
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Test Insert')),
      body: Center(
        child: ElevatedButton(
          onPressed: () {
            print('Inserting test data...');
            context.read<TransactionProvider>().addTransaction(
                  'เงินเดือน',
                  20000.0,
                  DateTime.now(),
                  TransactionType.income,
                );
          },
          child: const Text('Add Test Income'),
        ),
      ),
    );
  }
}
```

3. คำแนะนำ: ถอนการติดตั้งแอป แล้วรันใหม่ กดปุ่ม "Add Test Income"
4. ผลลัพธ์ที่คาดหวัง (Console): สังเกตลำดับของข้อความ ฐานข้อมูลยังไม่ถูกเปิดจนกว่าจะกดปุ่ม เพราะในกระบวนการนี้ยังไม่มีโค้ดใดเรียก _initDatabase() ตอนสร้าง provider และข้อความ Creating table มาก่อน Database initialized เพราะ onCreate ทำงานอยู่ภายใน openDatabase ก่อนที่ openDatabase จะคืนค่ากลับมา

```
I/flutter ( 1111): Inserting test data...
I/flutter ( 1111): Creating table transactions...
I/flutter ( 1111): Database initialized at
/data/user/0/com.example.expense_tracker_provider/databases/expenses.db
I/flutter ( 1111): Inserted transaction with id: 1
```

กระบวนการที่ 4: การอ่านข้อมูล (Read)

เป้าหมายของกระบวนการ

เพื่อเพิ่มความสามารถให้แอปสามารถ ดึงข้อมูลทั้งหมด ที่บันทึกไว้ในฐานข้อมูลออกมาและแสดงผลให้ผู้ใช้เห็นบนหน้าจอหลัก ทำให้ผู้ใช้สามารถเห็นภาพรวมของรายรับ-รายจ่ายทั้งหมดได้

- ความสามารถในแอป: หน้าจอหลัก (TransactionListScreen) จะแสดงรายการธุรกรรมทั้งหมดที่เคยบันทึกไว้
- การเชื่อมต่อ: TransactionProvider จะถูกสร้างเมื่อเริ่มแอปและเรียก fetchAndSetTransactions เพื่อดึงข้อมูล, TransactionListScreen จะใช้ Consumer เพื่อ "ฟัง" ข้อมูลจาก Provider และแสดงผล
- Input/Output: Input คือ การเรียกเมธอด fetchAndSetTransactions, Output คือ List<MyTransaction> ที่ถูกอัปเดตใน Provider ซึ่งส่งผลให้ UI แสดงผลข้อมูลนั้น

1. คำแนะนำ: เพิ่มเมธอด fetchAndSetTransactions ใน TransactionProvider

```
// ภายในคลาส TransactionProvider
// กระบวนการที่ 4: การ Read
Future<void> fetchAndSetTransactions() async {
  await _initDatabase();
  if (_database == null) return;

  final dataList = await _database!.query(_tableName, orderBy: 'date DESC');
  _transactions = dataList.map((item) => MyTransaction.fromMap(item)).toList();
  print('Fetched ${_transactions.length} transactions.');
  notifyListeners(); // แจ้ง UI ให้วาดใหม่
}
```

2. คำแนะนำ: เรียก fetchAndSetTransactions ใน constructor ของ Provider เพื่อให้ข้อมูลถูกโหลดเมื่อแอปเริ่ม

```
// ภายในคลาส TransactionProvider
class TransactionProvider with ChangeNotifier {
  //...
  List<MyTransaction> _transactions = [];

  TransactionProvider() {
    fetchAndSetTransactions(); // โหลดข้อมูลเมื่อ Provider ถูกสร้าง
  }
  //...
}
```

ระวังชื่อคลาสชนกัน: ชนิดของรายการต้องเป็น List<MyTransaction> ถ้าพิมพ์เป็น List<Transaction> โค้ดจะคอมไพล์ไม่ผ่าน เพราะ Transaction เป็นชื่อคลาสที่มีอยู่แล้วในแพ็กเกจ sqflite ซึ่งหมายถึง transaction ของฐานข้อมูลตามหัวข้อ 13.9 นี่คือเหตุผลที่ปฏิบัติการนี้ตั้งชื่อ Model ว่า MyTransaction ตั้งแต่ต้น

คำแนะนำ: กลับไปที่เมธอด addTransaction แล้วเปิดคอมเมนต์บรรทัด await fetchAndSetTransactions(); เพื่อให้รายการบนหน้าจออัปเดตทันทีหลังเพิ่มข้อมูล

3. การทดสอบ: สร้างหน้าจอหลัก screens/transaction_list_screen.dart และปรับปรุง main.dart

```
// สร้างไฟล์ lib/screens/transaction_list_screen.dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'package:intl/intl.dart';
import '../providers/transaction_provider.dart';
import '../models/my_transaction.dart';

class TransactionListScreen extends StatelessWidget {
  const TransactionListScreen({super.key});
  @override
  Widget build(BuildContext context) {
```

```
    return Scaffold(
      appBar: AppBar(title: const Text('รายรับ-รายจ่าย')),
      body: Consumer<TransactionProvider>(
        builder: (context, txProvider, child) =>
            txProvider.transactions.isEmpty
                ? const Center(child: Text('ไม่มีรายการ'))
                : ListView.builder(
                    itemCount: txProvider.transactions.length,
                    itemBuilder: (ctx, i) {
                      final tx = txProvider.transactions[i];
                      return ListTile(
                        leading: CircleAvatar(
                          child: Text(tx.type == TransactionType.income ? 'รับ' : '
จ่าย'),
                        ),
                        title: Text(tx.title),
                        subtitle: Text(DateFormat.yMMMd().format(tx.date)),
                        trailing: Text(
                          '${tx.amount.toStringAsFixed(2)} บาท',
                          style: TextStyle(
                            color: tx.type == TransactionType.income ?
Colors.green : Colors.red,
                          ),
                        ),
                      );
                    },
                  ),
      ),
      // ปุ่มเพิ่มรายการตัวอย่างชั่วคราว จนกว่าจะสร้างหน้าฟอร์มในการบ้าน
      floatingActionButton: FloatingActionButton(
        onPressed: () => context.read<TransactionProvider>().addTransaction(
              'ค่าอาหาร',
              120.0,
              DateTime.now(),
              TransactionType.expense,
            ),
        child: const Icon(Icons.add),
      ),
    );
  }
}
```

```
// lib/main.dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
```

```dart
import 'providers/transaction_provider.dart';
import 'screens/transaction_list_screen.dart';

void main() {
  runApp(
    ChangeNotifierProvider(
      create: (context) => TransactionProvider(),
      child: const MyApp(),
    ),
  );
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});
  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      title: 'Expense Tracker',
      home: TransactionListScreen(), // เปลี่ยนเป็นหน้าจอหลัก
    );
  }
}
```

4. คำแนะนำ: รันแอป (Hot Restart)
5. ผลลัพธ์ที่คาดหวัง (UI): คุณจะเห็นรายการ "เงินเดือน" ที่เพิ่มไว้ในขั้นตอนก่อนหน้าแสดงขึ้นมาบนหน้าจอ และเมื่อกดปุ่ม + จะมีรายการ "ค่าอาหาร" เพิ่มขึ้นทันที

กระบวนการที่ 5 และ 6: การแก้ไข (Update) และลบข้อมูล (Delete)

เป้าหมายของกระบวนการ

เพื่อทำให้แอปพลิเคชันมีการจัดการข้อมูลที่ครบวงจร โดยผู้ใช้สามารถ แก้ไขรายละเอียด ของรายการที่มีอยู่เดิม หรือ ลบรายการ ที่ไม่ต้องการออกจากระบบได้

- ความสามารถในแอป: ผู้ใช้สามารถแตะที่รายการเพื่อเปิดหน้าฟอร์มแก้ไข หรือกดปุ่มลบเพื่อนำรายการนั้นออกจากลิสต์
- การเชื่อมต่อ: TransactionListScreen จะเรียกเมธอด deleteTransaction, ในขณะที่ AddEditTransactionScreen (การบ้าน) จะเรียก updateTransaction ใน TransactionProvider. Provider จะส่งคำสั่ง UPDATE หรือ DELETE ไปยังฐานข้อมูล
- Input/Output: Input คือ id ของรายการที่ต้องการแก้ไข/ลบ และอาจมีข้อมูลใหม่ (สำหรับ Update), Output คือการที่ข้อมูลในฐานข้อมูลและ UI ถูกเปลี่ยนแปลงตามนั้น

1. คำแนะนำ: เพิ่มเมธอด updateTransaction และ deleteTransaction ใน TransactionProvider

```dart
// ภายในคลาส TransactionProvider
// กระบวนการที่ 5: การ Update
Future<void> updateTransaction(int id, MyTransaction newTransaction) async {
  await _initDatabase();
```

```
  if (_database == null) return;
  await _database!.update(
    _tableName,
    newTransaction.toMap(), // toMap() ไม่ส่ง id ที่เป็น null จึงไม่ไปเปลี่ยนคีย์หลัก
    where: 'id = ?',
    whereArgs: [id],
  );
  await fetchAndSetTransactions();
}

// กระบวนการที่ 6: การ Delete
Future<void> deleteTransaction(int id) async {
  await _initDatabase();
  if (_database == null) return;
  await _database!.delete(
    _tableName,
    where: 'id = ?',
    whereArgs: [id],
  );
  await fetchAndSetTransactions();
}
```

2. การทดสอบ: เพิ่มปุ่มลบใน ListTile ของ TransactionListScreen

```
// ใน ListTile ของ TransactionListScreen
// ...
trailing: Row(
  mainAxisSize: MainAxisSize.min,
  children: [
    Text(
      '${tx.amount.toStringAsFixed(2)} บาท',
      style: TextStyle(
        color: tx.type == TransactionType.income ? Colors.green : Colors.red,
      ),
    ),
    IconButton(
      icon: const Icon(Icons.delete, color: Colors.grey),
      onPressed: () {
        // เรียกเมธอด delete
        context.read<TransactionProvider>().deleteTransaction(tx.id!);
      },
    ),
  ],
),
// ...
```

3. คำแนะนำ: รันแอป (Hot Restart), กดปุ่ม + เพิ่มข้อมูลใหม่สัก 2-3 รายการ, จากนั้นลองกดปุ่มลบ

4. ผลลัพธ์ที่คาดหวัง: รายการที่ถูกลบจะหายไปจากหน้าจอทันที และเมื่อปิดเปิดแอปใหม่ รายการนั้นก็จะไม่กลับมา
5. การบ้านสำหรับผู้เรียน: ให้สร้างหน้า AddEditTransactionScreen ที่เป็นฟอร์มสำหรับเพิ่มและแก้ไขข้อมูล โดยใช้ความรู้จากบทที่ 8 และ 9 เพื่อส่งข้อมูลไปมาระหว่างหน้าจอ และเรียกใช้เมธอด addTransaction หรือ updateTransaction จาก Provider

ยินดีด้วย! คุณได้สร้างแอปพลิเคชันที่ใช้ Provider เป็นศูนย์กลางในการจัดการฐานข้อมูล SQLite อย่างสมบูรณ์ ซึ่งเป็นสถาปัตยกรรมที่สะอาดและง่ายต่อการขยายในอนาคต

ความท้าทายเพิ่มเติม

ข้อที่ 1 เพิ่มคอลัมน์ note สำหรับบันทึกรายละเอียดของแต่ละรายการ โดยเปลี่ยน version เป็น 2 และเขียน onUpgrade ตามหัวข้อ 13.8 ทดสอบโดยรันทับแอปเดิมที่มีข้อมูลอยู่แล้วโดยไม่ถอนการติดตั้ง รายการเดิมต้องยังอยู่ครบ

ข้อที่ 2 แสดงยอดคงเหลือไว้ใต้ AppBar โดยคำนวณในฐานข้อมูลด้วยคำสั่ง SUM ผ่าน rawQuery แทนการวนบวกใน Dart แล้วเทียบว่าต่างกันอย่างไรเมื่อมีรายการหลายพันแถว

ข้อที่ 3 เพิ่มปุ่มนำเข้ารายการตัวอย่างครั้งละ 100 รายการ ทำสองแบบคือเรียก addTransaction ทีละรายการ กับใช้ batch ภายใน transaction ตามหัวข้อ 13.9 แล้วจับเวลาเปรียบเทียบ